

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

TIỂU LUẬN HỌC PHẦN CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
HỆ THỐNG QUẢN LÝ HỒ SƠ SINH VIÊN VÀ ENGINE ĐỐI SÁNH HIỆU
NĂNG THUẬT TOÁN (DSA) – RECORDS-AND-DECISION ENGINE

Giảng viên hướng dẫn: ThS. Bảo Vũ Đình (T-Bao)

Mã lớp học phần: DASA230179_03

Nhóm thực hiện: Nhóm 12

Sinh viên thực hiện:

Họ và tên	MSSV
Bùi Tấn Phát (Nhóm trưởng)	25110289
Nguyễn Phan Thành Tiến	25110361
Ngô Đức Thành	25110335
Đặng Đoàn Minh Khang	25110227

TP. Hồ Chí Minh, tháng 09 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM



HCMUTE

TIỂU LUẬN HỌC PHẦN CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
HỆ THỐNG QUẢN LÝ HỒ SƠ SINH VIÊN VÀ ENGINE ĐỐI SÁNH HIỆU
NĂNG THUẬT TOÁN (DSA) – RECORDS-AND-DECISION ENGINE

Giảng viên hướng dẫn: ThS. Bảo Vũ Đình (T-Bao)

Mã lớp học phần: DASA230179_03

Nhóm thực hiện: Nhóm 12

Sinh viên thực hiện:

Họ và tên	MSSV
Bùi Tấn Phát (Nhóm trưởng)	25110289
Nguyễn Phan Thành Tiến	25110361
Ngô Đức Thành	25110335
Đặng Đoàn Minh Khang	25110227

TP. Hồ Chí Minh, tháng 09 năm 2026

LỜI CẢM ƠN

Lời đầu tiên, Nhóm 12 chúng em xin chân thành gửi lời cảm ơn sâu sắc nhất đến Ban Giám hiệu Trường Đại học Công nghệ Kỹ thuật TP. Hồ Chí Minh cùng toàn thể quý Thầy Cô đã tạo điều kiện học tập thuận lợi và trang bị cho chúng em những nền tảng tri thức quý báu.

Đặc biệt, chúng em xin gửi lời tri ân chân thành đến Thầy ThS. Bảo Vũ Đình (T-Bao) — giảng viên phụ trách học phần Cấu trúc Dữ liệu và Giải thuật (Mã lớp học phần: DASA230179_03). Trong suốt 14 tuần học tập vừa qua, Thầy đã luôn mang đến cho chúng em những bài giảng tâm huyết, sinh động và giàu tính thực tiễn. Sự đồng hành, tận tình hướng dẫn, lắng nghe và giải đáp mọi vướng mắc từ Thầy đã giúp chúng em thấu hiểu sâu sắc bản chất môn học cũng như định hướng rõ ràng trong suốt quá trình xây dựng đồ án.

Chúng em xin trân trọng tiếp thu mọi lời nhận xét, đóng góp quý giá từ Thầy để không ngừng hoàn thiện đồ án, trau dồi tư duy giải thuật và phát triển vững chắc các kỹ năng chuyên môn cho chặng đường phía trước. Kính chúc Thầy luôn dồi dào sức khỏe, tràn đầy niềm vui và luôn gặt hái được nhiều thành công trong sự nghiệp giảng dạy!

Một lần nữa, Nhóm 12 chúng em xin chân thành cảm ơn Thầy!

TP. Hồ Chí Minh, ngày 28 tháng 09 năm 2026

Sinh viên thực hiện

Nhóm 12

BẢNG ĐÁNH GIÁ VÀ PHÂN CÔNG NHIỆM VỤ

Mã lớp học phần: DASA230179_03

Nhóm thực hiện: Nhóm 12

Giảng viên hướng dẫn: ThS. Bảo Vũ Đình (T-Bao)

STT	Họ và tên / MSSV	Nhiệm vụ phân công	Mức độ hoàn thành
1	Bùi Tấn Phát (Nhóm trưởng) MSSV: 25110289	Điều phối nhóm, tổng hợp báo cáo, phụ trách yêu cầu MC1 : Cài đặt Bảng băm đóng (Closed Hash Table) với FNV-1a và Linear Probing, phân tích độ phức tạp.	100%
2	Nguyễn Phan Thành Tiến MSSV: 25110361	Phụ trách yêu cầu MC2 : Cài đặt Cây đồng cực đại (Custom Max-Heap) với Floyd Build Heap $O(N)$ và heapify-Down dạng lặp, thực hiện đo đặc benchmark.	100%
3	Ngô Đức Thành MSSV: 25110335	Phụ trách yêu cầu tự phát hiện RQ1 : Thiết kế và cài đặt bộ lọc sinh viên theo Lớp (Class Index View) với truy vấn trung bình $O(1)$.	100%
4	Đặng Đoàn Minh Khang MSSV: 25110227	Phụ trách yêu cầu tự phát hiện RQ2 : Cài đặt bộ lọc khoảng GPA bằng Binary Search 2 đầu biên trên mảng sắp xếp; thiết kế kiến trúc hệ thống và API Bridge.	100%

Đánh giá kết quả: Các thành viên hoàn thành các nhiệm vụ đúng thời hạn, hoàn thành nhiệm vụ tốt. Mặc dù có chút khó khăn trong quá trình tìm hiểu các kiến thức nhưng được sự hỗ trợ từ các bạn trong nhóm, cùng lớp đã hoàn thành đồ án đúng thời hạn.

TUYÊN BỐ QUYỀN SỞ HỮU THÀNH PHẦN - D3

Tuân thủ quy định về Đánh giá năng lực cá nhân theo Mục 10, 11 và 19 của Đề bài, từng thành viên xác nhận quyền sở hữu kỹ thuật, trách nhiệm cài đặt và lập luận thiết kế độc lập như sau:

Thành viên & MSSV	Thành phần sở hữu	Đóng góp Kỹ thuật & Biện minh Thiết kế
Bùi Tấn Phát (Nhóm trưởng) MSSV: 25110289	Yêu cầu MC1: Closed Hash (Tra cứu theo MSSV)	Tự cài đặt FNV-1a 64-bit, Linear Probing và cấu trúc ô nhớ 2 trạng thái (OCCUPIED/EMPTY) tính sau Bulk Load. Bảng băm chọn kích thước động $M = 2N + 1$ để giữ hệ số tải $\alpha \approx 0.50$; với benchmark 10.000.000 bản ghi thì $M = 20.000.001$.
Nguyễn Phan Thành Tiến MSSV: 25110361	Yêu cầu MC2: Custom Heap (Truy xuất GPA cao nhất)	Tự cài đặt Max-Heap mảng 1D, Floyd Build Heap $O(N)$ từ dưới lên, heapifyDown dạng lặp và xử lý quy tắc Tie-break theo MSSV. Giải thích trực quan độ phức tạp $O(N)$ dựa trên cấu trúc phân tầng của cây nhị phân.
Ngô Đức Thành MSSV: 25110335	Yêu cầu RQ1: Class Filter (Lọc theo mã Lớp)	Thiết kế và hiện thực ClassIndexFilter theo cơ chế Class Index View (unordered_map<string, vector<int>»), tách chi phí build $O(N)$ khỏi pha query trung bình $O(1)$, trả view kết quả mà không sao chép bản ghi Student.
Đặng Đoàn Minh Khang MSSV: 25110227	Yêu cầu RQ2: Sorted GPA & Kiến trúc hệ thống (Lọc khoảng GPA)	Xây dựng mảng con trở sắp xếp GPA bằng std::sort, tự cài đặt thuật toán Tìm kiếm Nhị phân 2 đầu biên (Lower/Upper Bound) tạo range view trong thời gian $O(\log N)$; thiết kế kiến trúc phân tầng 3 tầng, quy trình Bulk Load và API Bridge.

Cam kết cá nhân: Toàn bộ các thành viên cam kết hiểu sâu sắc mã nguồn thành phần do mình sở hữu, nắm vững lý thuyết độ phức tạp và sẵn sàng giải thích, trace code độc lập trong buổi vấn đáp (D8).

MỤC LỤC

PHẦN MỞ ĐẦU	1
1. Lý do chọn đề tài và bối cảnh thực tiễn của bài toán	1
2. Mục tiêu nghiên cứu và chuẩn đầu ra học phần (CLO Alignment)	1
3. Đối tượng, phạm vi nghiên cứu và tập dữ liệu thử nghiệm	2
4. Phương pháp tiếp cận, công cụ thực hiện và bố cục báo cáo	3
CHƯƠNG 1: TỔNG QUAN BÀI TOÁN VÀ THIẾT KẾ CẤU TRÚC	
DỮ LIỆU	4
1.1. Mô tả bài toán và phân tích nghiệp vụ hệ thống	4
1.1.1. Bối cảnh bài toán Records-and-Decision Engine	4
1.1.2. Mô hình hóa thực thể dữ liệu và các trường thuộc tính	4
1.1.3. Phân tích các mô thức truy cập dữ liệu và thao tác chủ đạo	5
1.2. Xác định và mô hình hóa các yêu cầu bắt buộc chung	6
1.2.1. Yêu cầu MC1: Tra cứu chính xác theo định danh ở quy mô lớn	6
1.2.2. Yêu cầu MC2: Truy xuất phần tử theo thứ tự và mức ưu tiên cực trị	6
1.3. Phân tích và phát hiện các yêu cầu đặc thù của lĩnh vực	7
1.3.1. Yêu cầu tự phát hiện 1: Lọc dữ liệu theo điều kiện chỉ mục	7
1.3.2. Yêu cầu tự phát hiện 2: Truy vấn dữ liệu theo khoảng giá trị	7
1.4. Phân tích xung đột yêu cầu và giải pháp xử lý kỹ thuật	8
1.4.1. Xác định cặp yêu cầu xung đột cốt lõi trong hệ thống	8
1.4.2. So sánh hướng Kết hợp cấu trúc và Đánh đổi có chủ đích	8
1.4.3. Biện minh phương án kỹ thuật giải quyết xung đột của nhóm	9
1.5. Áp dụng khung đánh giá 7 tiêu chí và quy trình lựa chọn Q1–Q4	9
1.5.1. Ma trận so sánh 7 tiêu chí kỹ thuật trên các cấu trúc ứng viên	9

1.5.2. Áp dụng quy trình Q1–Q4 cho yêu cầu tra cứu chính xác	
MC1	11
1.5.3. Áp dụng quy trình Q1–Q4 cho yêu cầu trích xuất cực trị	
MC2	12
1.5.4. Áp dụng quy trình Q1–Q4 cho các yêu cầu tự phát hiện . .	12
1.6. Biện minh chi tiết việc lựa chọn cấu trúc dữ liệu cốt lõi	13
1.6.1. Biện minh lựa chọn Bảng băm đóng (Closed Hash Table)	
cho MC1	13
1.6.2. Biện minh lựa chọn Cây đồng cực đại (Custom Max-Heap)	
cho MC2	13
1.6.3. Biện minh cấu trúc dữ liệu hỗ trợ cho các truy vấn theo	
khoảng	14
1.7. Phân tích và giải trình các phương án kỹ thuật bị loại bỏ	14
1.7.1. Lý do loại bỏ giải pháp duyệt tuyến tính trên Mảng tuần tự	14
1.7.2. Lý do loại bỏ Bảng băm mở (Chaining) và Danh sách liên	
kết	14
1.7.3. Lý do loại bỏ Cây tìm kiếm nhị phân thông thường (Un-	
balanced BST)	15
1.8. Thiết kế kiến trúc tổng thể 3 tầng của hệ thống	15
1.8.1. Tầng giao diện người dùng (Presentation Layer)	15
1.8.2. Tầng cấu trúc dữ liệu cốt lõi và thuật toán (DSA Core Layer)	15
1.8.3. Tầng lưu trữ bền vững (Persistence Layer) và tính phân tách	16
1.8.4. Cấu trúc tổ chức thư mục mã nguồn dự án	17

CHƯƠNG 2: HIỆN THỰC HỆ THỐNG, THỰC NGHIỆM VÀ ĐỐI SÁNH HIỆU NĂNG

2.1. Cài đặt chi tiết cấu trúc Bảng băm đóng từ đầu (From Scratch 1) .	18
2.1.1. Thiết kế ô nhớ và cấu trúc Entry	18
2.1.2. Hiện thực hàm băm chuỗi FNV-1a và phân bố giá trị băm .	18

2.1.3. Cơ chế giải quyết đưng độ và thuật toán tìm kiếm	20
2.1.4. Phân tích chi phí thời gian và không gian bộ nhớ	21
2.2. Cài đặt chi tiết cấu trúc Cây đồng cực đại từ đầu (From Scratch 2)	22
2.2.1. Biểu diễn mảng phẳng 1 chiều và công thức truy xuất chỉ số nút	22
2.2.2. Hiện thực thuật toán vun đống Floyd Build Heap trong thời gian $O(N)$	23
2.2.3. Cài đặt các thao tác truy xuất cực trị và cập nhật đống	25
2.2.4. Xử lý quy tắc phân định đồng hạng (Tie-breaking Rule)	25
2.3. Cài đặt các thuật toán hỗ trợ và xử lý Range Query	26
2.3.1. Xây dựng mảng dữ liệu sắp xếp theo điểm GPA (RQ2)	27
2.3.2. Hiện thực thuật toán tìm kiếm nhị phân 2 đầu biên trên mảng sắp xếp	27
2.3.3. Hiện thực bộ lọc theo mã lớp học phần (RQ1) bằng Class Index View	29
2.4. Hiện thực tầng lưu trữ bền vững và cơ chế nạp dữ liệu hàng loạt	30
2.4.1. Thiết kế cấu trúc lưu trữ file dữ liệu (JSON / CSV)	30
2.4.2. Cơ chế nạp dữ liệu hàng loạt (Bulk Load) vào bộ nhớ RAM	30
2.4.3. Đảm bảo nguyên tắc độc lập giữa lưu trữ và xử lý truy vấn	32
2.5. Xây dựng giao diện tương tác và trực quan hóa dữ liệu	32
2.5.1. Thiết kế giao diện dòng lệnh CLI tương tác nhanh	32
2.5.2. Xây dựng module trực quan hóa cấu trúc dữ liệu và benchmark	32
2.6. Thiết kế kịch bản kiểm thử tự động và xử lý trường hợp biên	32
2.6.1. Xây dựng bộ Unit Test tự động cho các cấu trúc tự cài đặt	32
2.6.2. Kiểm thử xử lý dữ liệu trùng lặp, dữ liệu rỗng và tràn ngưỡng	33
2.6.3. Kiểm thử tính ổn định khi chạy tải truy vấn liên tục	33

2.7. Thực nghiệm đo đặc Benchmark hiệu năng trên tập dữ liệu quy mô lớn	33
2.7.1. Môi trường phần cứng, hệ điều hành và phương pháp luận đo lường	33
2.7.2. Kết quả đo thời gian tra cứu MC1 qua các mốc dữ liệu lớn	36
2.7.3. Kết quả đo thời gian trích xuất cực trị MC2 và xây dựng đồng	38
2.7.4. Kết quả đo thời gian thực thi các truy vấn tự phát hiện . .	39
2.8. Đối sánh hiệu năng thực nghiệm với lý thuyết và thuật toán tuyến tính	40
2.8.1. So sánh tốc độ Bảng băm đóng ($O(1)$) với Quét tuyến tính ($O(N)$)	40
2.8.2. So sánh tốc độ truy xuất Max-Heap ($O(1)$) với Quét tìm Max tuyến tính ($O(N)$)	42
2.8.3. Đánh giá chi tiết mức độ tiêu hao bộ nhớ RAM và hiệu ứng Cache	42
2.9. Nhật ký gỡ lỗi (Debug Log) và tối ưu hóa giải thuật	43
2.9.1. Ghi nhận lỗi phát sinh khi xử lý điều kiện dừng tìm kiếm trên Bảng băm	43
2.9.2. Tối ưu hóa cấp phát bộ nhớ và bảo toàn con trỏ trong Bulk Load	43
2.10. Tuyên bố Trách nhiệm Giải trình về Sử dụng AI và Liêm chính Học thuật (AI-Audit Log)	44
PHẦN KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM	46
1. Kết luận tổng quan về dự án	46
2. Bài phản tư cá nhân của 4 thành viên (Individual Reflections - D7) .	46
2.1. Phản tư của sinh viên Bùi Tấn Phát (MSSV: 25110289) . . .	46

2.2. Phản tư của sinh viên Nguyễn Phan Thành Tiến (MSSV: 25110361)	47
2.3. Phản tư của sinh viên Ngô Đức Thành (MSSV: 25110335) .	47
2.4. Phản tư của sinh viên Đặng Đoàn Minh Khang (MSSV: 25110227)	48
TÀI LIỆU THAM KHẢO	49

DANH SÁCH HÌNH VẼ

Hình 1:	Cấu trúc thực thể Student	19
Hình 2:	Cấu trúc ô nhớ Entry trong Bảng băm	19
Hình 3:	Mã nguồn hàm băm chuỗi FNV-1a 64-bit tự cài đặt	19
Hình 4:	Mã nguồn thuật toán nạp dữ liệu và xử lý đụng độ tuyến tính trong Bảng băm	20
Hình 5:	Mã nguồn giải thuật tìm kiếm tra cứu chính xác theo MSSV trên Bảng băm	21
Hình 6:	Sơ đồ tuần tự (Sequence Diagram) luồng tra cứu chính xác theo MSSV trên Bảng băm	22
Hình 7:	Mã nguồn thuật toán Floyd Build Heap $O(N)$ tự cài đặt	23
Hình 8:	Mã nguồn hàm heapifyDown duy trì bất biến cấu trúc Max-Heap	25
Hình 9:	Sơ đồ tuần tự (Sequence Diagram) luồng trích xuất GPA cao nhất trên Max-Heap	26
Hình 10:	Hiện thực toán tử phân định đồng hạng kết hợp GPA và MSSV	26
Hình 11:	Mã nguồn xây dựng mảng dữ liệu sắp xếp theo GPA phục vụ Range Query	27
Hình 12:	Mã nguồn thuật toán Binary Search 2 đầu biên trích xuất khoảng GPA	28
Hình 13:	Sơ đồ tuần tự (Sequence Diagram) luồng lọc sinh viên theo khoảng GPA	29
Hình 14:	Sơ đồ tuần tự (Sequence Diagram) luồng khởi tạo và nạp dữ liệu hàng loạt (Bulk Load)	31
Hình 15:	Benchmark MC1 tra cứu MSSV tại đầu dải dữ liệu	37

Hình 16:	Benchmark MC1 tra cứu MSSV tại giữa dải dữ liệu	37
Hình 17:	Benchmark MC1 tra cứu MSSV tại cuối dải dữ liệu	37
Hình 18:	Benchmark MC1 tra cứu MSSV khi khóa không tồn tại trong hệ thống	38
Hình 19:	Biểu đồ Benchmark MC2 trích xuất GPA cao nhất trên Max-Heap so với Baseline	38
Hình 20:	Benchmark RQ1 lọc danh sách sinh viên lớp C01 (Đầu dải)	39
Hình 21:	Benchmark RQ1 lọc danh sách sinh viên lớp C13 (Giữa dải)	39
Hình 22:	Benchmark RQ1 lọc danh sách sinh viên lớp C25 (Cuối dải)	40
Hình 23:	Benchmark RQ1 khi tra cứu mã lớp không tồn tại	40
Hình 24:	Dashboard minh họa truy vấn RQ2 với khoảng GPA [8.0, 9.5]; số liệu C++ Core trong Bảng 3 sử dụng workload [8.0, 10.0]	41

DANH MỤC TỪ VIẾT TẮT

Ký hiệu	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
DSA	Data Structures and Algorithms	Cấu trúc Dữ liệu và Giải thuật
MSSV	Student Identifier	Mã số sinh viên (Khóa chính định danh)
GPA	Grade Point Average	Điểm trung bình tích lũy học tập
MC	Mandatory Criteria	Yêu cầu nghiệp vụ bắt buộc cốt lõi
MC1	Mandatory Criteria 1	Tra cứu chính xác sinh viên theo MSSV
MC2	Mandatory Criteria 2	Truy xuất sinh viên có GPA cao nhất
RQ	Required Query	Yêu cầu truy vấn mở rộng tự phát hiện
RQ1	Requirement 1	Truy xuất danh sách sinh viên theo Lớp
RQ2	Requirement 2	Truy xuất sinh viên theo khoảng GPA
FNV-1a	Fowler–Noll–Vo (Version 1a)	Giải thuật hàm băm phi mã hóa tốc độ cao
RAM	Random Access Memory	Bộ nhớ truy xuất ngẫu nhiên khả biến
CPU	Central Processing Unit	Bộ vi xử lý trung tâm
JSON	JavaScript Object Notation	Định dạng dữ liệu hoán đổi văn bản chuẩn
STL	Standard Template Library	Thư viện mẫu chuẩn trong ngôn ngữ C++
CLO	Course Learning Outcomes	Chuẩn đầu ra tích hợp của học phần
CRUD	Create, Read, Update, Delete	Bốn thao tác dữ liệu cơ bản

PHẦN MỞ ĐẦU

1. Lý do chọn đề tài và bối cảnh thực tiễn của bài toán

Trong công tác quản lý và khai thác thông tin học tập tại các cơ sở giáo dục đại học quy mô lớn, việc lưu trữ, tra cứu và xử lý nhanh chóng khối lượng hồ sơ sinh viên ngày càng trở nên cấp thiết. Hệ thống thông tin quản lý không chỉ dừng lại ở chức năng lưu trữ dữ liệu tĩnh, mà còn cần hoạt động như một hệ thống xử lý hồ sơ và hỗ trợ ra quyết định (*Records-and-Decision Engine*) có khả năng phản hồi tức thì các tác vụ vận hành cốt lõi: tra cứu thông tin sinh viên theo mã định danh duy nhất, xếp hạng và truy xuất sinh viên có kết quả học tập cao nhất để xét duyệt học bổng, cũng như phân loại và thống kê danh sách sinh viên theo các tiêu chí điểm số hoặc lớp học.

Khi quy mô dữ liệu mở rộng lên mức thiết kế lớn gồm 10.000.000 sinh viên, các phương pháp duyệt tuần tự tuyến tính ($O(N)$) bộc lộ hạn chế rõ rệt về độ trễ, gây chiếm dụng tài nguyên CPU và làm giảm tốc độ phản hồi. Việc nghiên cứu và ứng dụng các cấu trúc dữ liệu chuyên sâu trong bộ nhớ chính (In-memory Data Structures) như Bảng băm đóng (Closed Hash Table) và Cây đồng cực đại (Max-Heap) là giải pháp quan trọng nhằm tối ưu thời gian xử lý một số tác vụ trọng yếu xuống mức $O(1)$ hoặc $O(\log N)$ tùy theo loại thao tác và cấu trúc dữ liệu được sử dụng. Xuất phát từ yêu cầu thực tiễn đó, Nhóm 12 lựa chọn đề tài: **"Hệ thống Quản lý Hồ sơ Sinh viên và Engine Đối sánh Hiệu năng Thuật toán (DSA)"** nhằm xây dựng giải pháp kỹ thuật hoàn chỉnh, đáp ứng cân bằng giữa hiệu năng thời gian và không gian bộ nhớ trên tập dữ liệu thực nghiệm hiện tại 10.000.000 bản ghi.

2. Mục tiêu nghiên cứu và chuẩn đầu ra học phần (CLO Alignment)

Đồ án được thiết kế bám sát các chuẩn đầu ra tích hợp của học phần Cấu trúc Dữ liệu và Giải thuật (Mã lớp học phần: DASA230179_03), tập trung vào 4 trọng tâm kỹ thuật:

Nhóm tiến hành phân tích bối cảnh và mô hình hóa dữ liệu bài toán thực

tế (CLO1), xác định rõ ràng đặc tả đầu vào–đầu ra, các ràng buộc dữ liệu lớn và thiết kế bao phủ toàn bộ các trường hợp biên đặc biệt (Edge Cases).

Nhóm tự tay lập trình từ đầu (*from scratch*) hai cấu trúc dữ liệu trọng tâm bằng C++20 (CLO3): Bảng băm đóng sử dụng hàm băm FNV-1a kết hợp giải thuật dò tuyến tính (Linear Probing), cùng Cây đồng cực đại (Custom Binary Max-Heap) với thuật toán vun đồng Floyd Build-Heap $O(N)$, hoàn toàn không sử dụng các cấu trúc dựng sẵn của thư viện STL.

Nhóm phân tích chi tiết cơ chế vận hành của cấu trúc cây phân cấp (CLO7), đánh giá điều kiện bảo toàn tính chất đồng và các trường hợp có thể gây suy giảm hiệu năng trong quá trình vận hành thực tế.

Nhóm áp dụng khung so sánh 7 tiêu chí kỹ thuật và quy trình 4 câu hỏi quyết định Q1–Q4 (CLO8) để giải trình tường minh phương án lựa chọn cấu trúc, làm rõ các điểm đánh đổi kỹ thuật (*trade-offs*) và bảo vệ thiết kế trước các phương án thay thế.

3. Đối tượng, phạm vi nghiên cứu và tập dữ liệu thử nghiệm

Về đối tượng nghiên cứu, đề tài tập trung vào các cấu trúc dữ liệu cơ bản và nâng cao (Mảng động, Bảng băm đóng, Cây đồng nhị phân, Mảng sắp xếp) cùng các giải thuật tìm kiếm, sắp xếp và tối ưu hóa truy vấn trong bộ nhớ RAM.

Về phạm vi nghiên cứu, hệ thống tập trung xử lý các thao tác gồm: tra cứu chính xác theo MSSV (MC1), tìm sinh viên có GPA cao nhất (MC2), lọc sinh viên theo Mã Lớp (RQ1) và lọc danh sách sinh viên theo khoảng GPA (RQ2).

Về tập dữ liệu thử nghiệm, lần đo C++ Core hiện tại nạp data/database.json làm dữ liệu gốc, sau đó bulk-add trong RAM để đạt 10.000.000 bản ghi sinh viên. Đây là quy mô đủ lớn để quan sát rõ rệt sự khác biệt giữa các đường đi thuật toán.

4. Phương pháp tiếp cận, công cụ thực hiện và bố cục báo cáo

Về phương pháp tiếp cận, nghiên cứu kết hợp giữa lý thuyết phân tích độ phức tạp thời gian/không gian (O, Ω, Θ) với phương pháp thực nghiệm đo đặc Benchmark bằng đồng hồ đo thời gian độ phân giải cao của thư viện chuẩn C++ (`std::chrono::high_resolution_clock`).

Về công cụ và môi trường phát triển, tầng xử lý thuật toán cốt lõi được xây dựng bằng ngôn ngữ C++ (chuẩn C++20). Tầng giao diện và trực quan hóa dữ liệu được xây dựng bằng React/Next.js kết hợp Ant Design, giao tiếp với C++ Engine thông qua giao thức API cầu nối (DSA Bridge).

Về bố cục báo cáo, tài liệu được tổ chức thành 2 chương trọng tâm cùng danh mục tài liệu tham khảo: Chương 1 trình bày tổng quan bài toán, mô hình hóa dữ liệu, phân tích xung đột yêu cầu và áp dụng quy trình Q1–Q4 để biện minh cấu trúc. Chương 2 trình bày chi tiết hiện thực mã nguồn C++ của hai cấu trúc cốt lõi từ đầu, kiến trúc phân tầng, kiểm thử trường hợp biên và toàn bộ số liệu Benchmark thực nghiệm đối sánh trên tập dữ liệu hiện tại 10.000.000 sinh viên, kèm nhật ký gỡ lỗi và danh mục tài liệu tham khảo chuyên ngành.

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN VÀ THIẾT KẾ CẤU TRÚC DỮ LIỆU

1.1. Mô tả bài toán và phân tích nghiệp vụ hệ thống

1.1.1. Bối cảnh bài toán Records-and-Decision Engine

Trong công tác quản lý đào tạo đại học, các hệ thống thông tin phải tiếp nhận và xử lý khối lượng lớn dữ liệu phát sinh theo từng học kỳ. Hệ thống quản lý hồ sơ sinh viên cần đáp ứng vai trò của một hệ thống xử lý hồ sơ và hỗ trợ ra quyết định (*Records-and-Decision Engine*) trong thời gian thực. Hệ thống này trực tiếp giải quyết các bài toán nghiệp vụ thường gặp: tra cứu nhanh hồ sơ cá nhân của sinh viên theo mã số, trích xuất danh sách sinh viên có thành tích học tập cao nhất để xét khen thưởng, cũng như phân loại học lực sinh viên theo các dải điểm phục vụ công tác theo dõi kết quả học tập.

Khi dung lượng dữ liệu sinh viên đạt mức 10.000.000 bản ghi, các phương pháp duyệt tuần tự tuyến tính ($O(N)$) bộc lộ rõ sự suy giảm hiệu năng, gây lãng phí chu kỳ CPU và kéo dài thời gian phản hồi. Vì vậy, việc thiết kế và cài đặt các cấu trúc dữ liệu phù hợp trong bộ nhớ chính (In-memory Data Structures) đóng vai trò quan trọng nhằm tối ưu thời gian xử lý một số truy vấn trọng yếu, có thể đạt mức $O(1)$ hoặc $O(\log N)$ tùy theo loại thao tác và cấu trúc dữ liệu được sử dụng.

1.1.2. Mô hình hóa thực thể dữ liệu và các trường thuộc tính

Mỗi hồ sơ sinh viên trong hệ thống được định nghĩa qua cấu trúc thực thể Student gồm 4 trường dữ liệu thiết yếu, bám sát nghiệp vụ quản lý đào tạo:

Trường mã định danh sinh viên (id – MSSV) là chuỗi 8 chữ số đóng vai trò khóa chính duy nhất (*Unique Primary Key*), nằm trong dải giá trị từ 25150001 đến 35150000 trên tập dữ liệu benchmark 10.000.000 bản ghi.

Trường họ và tên (name) lưu trữ chuỗi họ tên đầy đủ của sinh viên.

Trường mã lớp (classId) thể hiện lớp sinh hoạt/học phần trực thuộc. Trong tập dữ liệu gốc 500.000 bản ghi, mã lớp phân bổ trên 25 lớp từ C01 đến C25;

khi sinh thêm dữ liệu để đạt mốc 10.000.000 bản ghi phục vụ đo tải lớn, hệ thống tạo thêm các lớp như 23DTH01, 25KTP02, ... giúp đa dạng hóa miền dữ liệu thử nghiệm.

Trường điểm tích lũy (gpa) là số thực dấu phẩy động độ chính xác kép (double) trong thang điểm từ 0.0 đến 10.0 (phân bố thực nghiệm tập trung từ 4.0 đến 10.0). Cấu trúc thực thể này được tổ chức gọn gàng để tối ưu hóa không gian lưu trữ và tăng hiệu quả bộ nhớ đệm khi nạp hàng loạt vào RAM.

1.1.3. Phân tích các mô thức truy cập dữ liệu và thao tác chủ đạo

Dựa trên phân tích hành vi vận hành thực tế, mô thức truy cập dữ liệu (Access Patterns) của hệ thống được chia thành hai nhóm thao tác rõ rệt: thao tác ghi (*Write Path*) và thao tác đọc (*Read Path*).

Thao tác ghi trong phạm vi benchmark diễn ra tập trung ở giai đoạn nạp dữ liệu ban đầu (*Bulk Loading*) với khối lượng thực nghiệm hiện tại 10.000.000 bản ghi trong bộ nhớ chính. Các phép đo MC1, MC2, RQ1 và RQ2 được thực hiện trên ảnh chụp dữ liệu đã nạp xong; phần CRUD của Web Dashboard là chức năng minh họa quản lý dữ liệu và được tách khỏi tập số liệu hiệu năng. Mốc 10.000.000 bản ghi được dùng trực tiếp cho ảnh Dashboard và bảng số benchmark cập nhật.

Ngược lại, hệ thống được thiết kế theo mô hình read-heavy, trong đó các thao tác đọc chiếm tỷ trọng chủ yếu trong quá trình vận hành, diễn ra liên tục với tần suất cao và đòi hỏi phản hồi tức thì. Các thao tác đọc chủ đạo bao gồm: tra cứu chính xác thông tin một sinh viên theo MSSV (MC1), truy xuất ngay lập tức sinh viên có điểm GPA cao nhất (MC2), lọc sinh viên theo mã lớp (RQ1) và trích xuất danh sách sinh viên theo khoảng điểm GPA (RQ2). Do đó, mục tiêu thiết kế là tối ưu hóa đường đi truy vấn của các thao tác đọc.

1.2. Xác định và mô hình hóa các yêu cầu bắt buộc chung

1.2.1. Yêu cầu MC1: Tra cứu chính xác theo định danh ở quy mô lớn

Yêu cầu bắt buộc thứ nhất (MC1 – *Exact Match Lookup*) đòi hỏi hệ thống phải tiếp nhận một khóa tìm kiếm là MSSV và trả về chính xác, toàn vẹn toàn bộ hồ sơ sinh viên tương ứng, hoặc thông báo không tìm thấy nếu khóa không tồn tại trong cơ sở dữ liệu.

Đặc tả kỹ thuật của yêu cầu MC1 xác định: Đầu vào là chuỗi id cần tra cứu; Đầu ra là con trỏ hoặc tham chiếu đến đối tượng Student thỏa mãn; Ràng buộc hiệu năng bắt buộc là thời gian thực thi phải đạt mức tiệm cận trung bình $O(1)$ độc lập với kích thước dữ liệu $N = 10.000.000$. Các trường hợp biên (Edge Cases) cần xử lý đầy đủ bao gồm việc tra cứu khóa nằm ở đầu dải dữ liệu (25150001), giữa dải dữ liệu (khoảng 30150000), cuối dải dữ liệu (35150000) và các trường hợp tra cứu khóa hoàn toàn không tồn tại trong hệ thống.

1.2.2. Yêu cầu MC2: Truy xuất phần tử theo thứ tự và mức ưu tiên cực trị

Yêu cầu bắt buộc thứ hai (MC2 – *Extreme Priority Retrieval*) đòi hỏi hệ thống phải trích xuất ngay lập tức thông tin của sinh viên đạt điểm trung bình tích lũy (gpa) cao nhất toàn trường nhằm phục vụ hội đồng thi đua khen thưởng và xét cấp học bổng.

Đặc tả kỹ thuật của yêu cầu MC2 xác định: Đầu vào là yêu cầu trích xuất cực đại toàn cục; Đầu ra là đối tượng sinh viên có gpa lớn nhất; Quy tắc giải quyết đồng hạng (*Tie-breaking Rule*) được nhóm quy định nghiêm ngặt: nếu có nhiều sinh viên cùng đạt mức điểm GPA cao nhất (ví dụ cùng đạt 10.0), hệ thống sẽ ưu tiên chọn sinh viên có MSSV nhỏ hơn theo thứ tự từ điển; Ràng buộc hiệu năng là thao tác lấy phần tử cực trị phải đạt thời gian $O(1)$ mà không được phép duyệt lại toàn bộ 10.000.000 phần tử hay thực hiện thao tác sắp xếp lại toàn bộ tập dữ liệu với chi phí $O(N \log N)$.

1.3. Phân tích và phát hiện các yêu cầu đặc thù của lĩnh vực

1.3.1. Yêu cầu tự phát hiện 1: Lọc dữ liệu theo điều kiện chỉ mục

Từ góc độ nghiệp vụ quản lý đào tạo thực tế, ban chủ nhiệm khoa và cố vấn học tập luôn cần truy xuất danh sách toàn bộ sinh viên thuộc một lớp học cụ thể (Yêu cầu RQ1 – *Filter by Class ID*).

Đặc thù của yêu cầu này là trường khóa lọc `classId` có tính phân nhóm cao nhưng số lượng nhóm hữu hạn. Trong tập benchmark gốc, 500.000 sinh viên được phân bổ trên 25 lớp; khi mở rộng/bulk-add lên 10.000.000 bản ghi, các lớp sinh ra thêm làm tăng mạnh kích thước nhóm cần lọc. Trong code hiện tại, RQ1 build trước chỉ mục `classId` -> indexes; vì vậy truy vấn theo lớp không cần quét lại toàn bộ danh sách và không sao chép các đối tượng Student. Chi phí build là $O(N)$, còn query trung bình là $O(1)$ để lấy view và chỉ phát sinh $O(K)$ khi tăng gọi cần liệt kê hoặc serialize kết quả.

1.3.2. Yêu cầu tự phát hiện 2: Truy vấn dữ liệu theo khoảng giá trị

Hội đồng xét học bổng, phòng đào tạo và các đơn vị kiểm định thường xuyên cần trích xuất danh sách sinh viên theo các dải điểm học lực biến động (Yêu cầu RQ2 – *Range Query by GPA*), ví dụ dải điểm Xuất sắc [9.0, 10.0], Giỏi [8.0, 8.9], hoặc Khá [6.5, 7.9].

Đây là bài toán truy vấn theo khoảng trên trường giá trị liên tục gpa. Cấu trúc dữ liệu phục vụ yêu cầu này bắt buộc phải duy trì được thứ tự tự nhiên của điểm số, cho phép xác định vị trí biên dưới (`minGPA`) và biên trên (`maxGPA`) trong thời gian logarit $O(\log N)$. Nếu chỉ cần tạo range view thì truy vấn dừng tại $O(\log N)$; nếu cần duyệt/hiển thị toàn bộ K sinh viên thỏa mãn thì tổng chi phí trở thành $O(\log N + K)$.

1.4. Phân tích xung đột yêu cầu và giải pháp xử lý kỹ thuật

1.4.1. Xác định cặp yêu cầu xung đột cốt lõi trong hệ thống

Trong quá trình thiết kế hệ thống, nhóm nhận diện một xung đột kỹ thuật nền tảng giữa hai nhóm yêu cầu: Yêu cầu tra cứu chính xác theo khóa ngẫu nhiên (MC1) và Yêu cầu truy xuất theo thứ tự ưu tiên hoặc theo khoảng giá trị liên tục (MC2 và RQ2).

Bản chất của Bảng băm (Hash Table) là sử dụng hàm băm để phân tán ngẫu nhiên các khóa vào các ô nhớ nhằm giảm thiểu đụng độ và đạt tốc độ tra cứu $O(1)$ trung bình. Tuy nhiên, chính cơ chế này phá hủy hoàn toàn thứ tự tự nhiên của dữ liệu, khiến Bảng băm hoàn toàn không thể thực hiện các thao tác tìm kiếm giá trị lớn nhất hay truy vấn theo khoảng nếu không duyệt qua toàn bộ M ô nhớ ($O(N)$). Ngược lại, các cấu trúc duy trì thứ tự như Cây nhị phân tìm kiếm cân bằng (AVL, Red-Black) hoặc Mảng sắp xếp (Sorted Array) cho phép giải quyết tốt bài toán cực trị và truy vấn khoảng ($O(\log N + K)$), nhưng lại không đồng thời bảo đảm tra cứu MSSV ở mức $O(1)$ và chi phí chèn/xóa lên $O(N)$ đối với mảng.

1.4.2. So sánh hướng Kết hợp cấu trúc và Đánh đổi có chủ đích

Nhóm đã tiến hành phân tích và so sánh hai hướng tiếp cận giải quyết xung đột theo chuẩn kiến trúc:

Hướng tiếp cận 1 (Đánh đổi có chủ đích – *Deliberate Trade-off*): Chỉ lựa chọn duy nhất một cấu trúc dữ liệu (chẳng hạn mảng sắp xếp toàn bộ) để phục vụ mọi yêu cầu. Hướng đi này giúp tiết kiệm bộ nhớ do chỉ duy trì một bản sao dữ liệu, nhưng buộc hệ thống phải chấp nhận đánh đổi hiệu năng tra cứu khi tìm kiếm trên thuộc tính không phải là khóa sắp xếp chính, gây suy giảm hiệu năng khi số lượng truy vấn MC1 tăng cao.

Hướng tiếp cận 2 (Kết hợp cấu trúc – *Composition Approach*): Thiết kế hệ thống đa cấu trúc đồng bộ, trong đó mỗi cấu trúc dữ liệu chuyên biệt đảm nhận đúng bài toán sở trường của mình. Trong cài đặt hiện tại, HashTable lưu con trỏ

const Student* trở về vùng dữ liệu gốc để trả lại bản ghi nhanh; SortedGpaFilter cũng lưu mảng con trở đã sắp xếp theo GPA và trả về range view, nhờ đó tránh sao chép Student khi lọc khoảng; riêng CustomMaxHeapGpaFinder duy trì bản sao vector<Student> nội bộ để đơn giản hóa quá trình vun đống.

1.4.3. Biện minh phương án kỹ thuật giải quyết xung đột của nhóm

Nhóm 12 quyết định lựa chọn giải pháp **Kết hợp cấu trúc chuyên biệt (Composition Approach)**. Cụ thể, hệ thống tích hợp bốn cấu trúc/chỉ mục hỗ trợ trong bộ nhớ: Bảng băm đóng (Closed Hash Table) phục vụ MC1 đạt $O(1)$ trung bình; Cây đống cực đại (Custom Binary Max-Heap) phục vụ MC2 đạt $O(1)$ khi đọc nút gốc sau khi đã build; Class Index View phục vụ RQ1 bằng ánh xạ classId -> vector<int> với query trung bình $O(1)$; và Mảng con trở sắp xếp theo GPA (Sorted Pointer Array) phục vụ RQ2 đạt $O(\log N)$ cho pha tìm biên và $O(1)$ để tạo view kết quả. Thiết kế này chấp nhận đánh đổi bộ nhớ: HashTable và SortedGPA tiết kiệm bằng cách lưu con trở về dữ liệu gốc, ClassIndex lưu chỉ số sinh viên theo lớp, trong khi Heap sao chép Student để cài đặt thao tác vun đống rõ ràng hơn.

1.5. Áp dụng khung đánh giá 7 tiêu chí và quy trình lựa chọn Q1–Q4

1.5.1. Ma trận so sánh 7 tiêu chí kỹ thuật trên các cấu trúc ứng viên

Nhóm tiến hành so sánh bốn cấu trúc dữ liệu ứng viên theo 7 tiêu chí kỹ thuật liên quan trực tiếp đến các yêu cầu của hệ thống: Mảng thô tuần tự (*Raw Dynamic Array*), Bảng băm đóng (*Closed Hash Table*), Cây đống cực đại (*Binary Max-Heap*) và Mảng dữ liệu đã sắp xếp (*Sorted Array*).

Ý nghĩa và vai trò của từng tiêu chí kỹ thuật trong hệ thống được xác định cụ thể như sau:

Truy xuất theo khóa định danh (*Key Lookup*): Đánh giá khả năng tìm kiếm chính xác hồ sơ sinh viên khi biết trước MSSV. Đây là thao tác trọng yếu của yêu cầu MC1, đòi hỏi độ phức tạp tiệm cận lý tưởng đạt $O(1)$ trung bình.

Truy xuất theo vị trí (*Positional Access*): Đánh giá tốc độ truy cập một

phần tử khi biết trước chỉ số ô nhớ trong mảng. Các cấu trúc dựa trên mảng phẳng liên tục có ưu thế tuyệt đối $O(1)$ nhờ cơ chế tính toán địa chỉ offset trực tiếp trên thanh ghi phần cứng.

Chi phí chèn và xóa (*Insertion and Deletion*): Đo lường chi phí cập nhật cấu trúc dữ liệu khi tiếp nhận thêm hồ sơ mới hoặc xóa bỏ bản ghi. Các cấu trúc như Đồng hoặc Bảng băm có chi phí từ $O(1)$ đến $O(\log N)$, trong khi mảng sắp xếp đòi hỏi chi phí dịch chuyển $O(N)$.

Duy trì thứ tự toàn phần (*Total Order*): Đánh giá mức độ bảo toàn trật tự sắp xếp tự nhiên giữa các phần tử theo một trường thuộc tính (như điểm GPA) mà không cần sắp xếp lại toàn bộ tập dữ liệu ($O(N \log N)$).

Truy vấn theo khoảng giá trị (*Range Query*): Đánh giá khả năng trích xuất nhanh chóng tập hợp phần tử thỏa mãn điều kiện nằm trong dải giá trị $[\min, \max]$, quyết định trực tiếp hiệu năng phục vụ yêu cầu RQ2.

Trích xuất phần tử cực trị (*Extreme Priority Retrieval*): Đo lường độ trễ khi hệ thống cần lấy ra phần tử có giá trị lớn nhất hoặc nhỏ nhất toàn cục. Cây đồng chiếm ưu thế khi luôn duy trì phần tử cực trị tại nút gốc với chi phí $O(1)$.

Độ ổn định và hành vi tiệm cận (*Asymptotic Stability*): Đánh giá mức độ sai khác giữa chi phí trong trường hợp trung bình và trường hợp xấu nhất nhằm loại trừ các điểm nghẽn nghiêm trọng có thể làm treo hệ thống.

Ma trận so sánh tổng hợp chi tiết 7 tiêu chí kỹ thuật trên 4 cấu trúc ứng viên được trình bày trong bảng dưới đây:

Tiêu chí	Mảng thô	Closed Hash	Max-Heap	Mảng Sort (GPA)	Cấu trúc tối ưu
1. Khóa (Key)	$O(N)$	$O(1)$ tb	$O(N)$	$O(N)$	Bảng băm (MC1)
2. Vị trí	$O(1)$	—	$O(1)$	$O(1)$	Mảng thô / Sort
3. Chèn / Xóa	$O(1) / O(N)$	Build/chèn $O(1)$ tb; delete không hỗ trợ trực tiếp trong index tĩnh	$O(\log N)$	$O(N)$	Heap / Hash
4. Thứ tự	$O(N \log N)$	—	$O(N \log N)$	$O(N)$	Mảng sắp xếp
5. Khoảng	$O(N)$	—	—	$O(\log N)$ view / $O(\log N + K)$ liệt kê	Mảng con trỏ sắp xếp (RQ2)
6. Cực trị	$O(N)$	$O(N)$	$O(1)$ đỉnh	$O(1)$ biên	Max-Heap (MC2)
7. Độ ổn định	$O(N)$	$O(1)/O(N)$	$O(\log N)$	$O(\log N)$	Phối hợp đa cấu trúc

Bảng 1. Ma trận so sánh 7 tiêu chí kỹ thuật trên 4 cấu trúc dữ liệu ứng viên

1.5.2. Áp dụng quy trình Q1–Q4 cho yêu cầu tra cứu chính xác MC1

Khi phân tích yêu cầu MC1 qua quy trình 4 câu hỏi quyết định (Q1–Q4 Framework):

Về yêu cầu trật tự (Q1): Thao tác tra cứu theo MSSV hoàn toàn không đòi hỏi duy trì thứ tự giữa các phần tử, chỉ cần tìm kiếm chính xác theo khóa.

Về đặc trưng khóa và khối lượng công việc (Q2): Khóa tra cứu là chuỗi số 8 chữ số phân bố đồng đều, vận hành trong mô hình đọc chiếm ưu thế (Read-Heavy) sau khi nạp dữ liệu.

Về kiểm soát rủi ro tiệm cận (Q3): Nguy cơ đọng độ băm và suy thoái hiệu năng được triệt tiêu hiệu quả nhờ độ phân tán cao của hàm băm FNV-1a

64-bit và việc duy trì hệ số tải thấp $\alpha \approx 0.50$.

Về tối ưu hóa phần cứng (Q4): Bảng băm đóng dạng mảng phẳng liên tục tận dụng tối đa tính định vị không gian của bộ đệm CPU Cache Line 64-byte, giảm thiểu đáng kể Cache Miss so với danh sách liên kết.

1.5.3. Áp dụng quy trình Q1–Q4 cho yêu cầu trích xuất cực trị MC2

Đối với yêu cầu MC2, quy trình câu hỏi quyết định làm rõ phương án thiết kế như sau:

Về yêu cầu trật tự (Q1): Nghiệp vụ tìm sinh viên có GPA cao nhất chỉ cần duy trì trật tự cục bộ tại nút gốc (phần tử cực đại), hoàn toàn không cần sắp xếp toàn phần cả 10.000.000 bản ghi.

Về tần suất và khối lượng công việc (Q2): Tần suất truy vấn cực trị diễn ra liên tục trong quá trình vận hành, trong khi việc xây dựng đồng chỉ cần thực hiện đúng một lần ở pha Bulk Load.

Về độ phức tạp tiệm cận (Q3): Thao tác trích xuất phần tử cực đại tại gốc đạt tốc độ tức thì $O(1)$; nếu có phát sinh cập nhật đồng thì chi phí phục hồi cũng chỉ ở mức logarit $O(\log N)$ rất ổn định.

Về tối ưu hóa phần cứng (Q4): Cấu trúc Cây đồng nhị phân được lưu trữ trực tiếp trên mảng một chiều liên tục, không tốn bộ nhớ lưu con trỏ cây (*Left, Right*), cho phép tính toán chỉ số cha–con bằng các phép toán số học $O(1)$ siêu tốc.

1.5.4. Áp dụng quy trình Q1–Q4 cho các yêu cầu tự phát hiện

Đối với yêu cầu RQ2 (Lọc sinh viên theo khoảng GPA): Câu hỏi Q1 xác định dữ liệu bắt buộc phải được sắp xếp theo thứ tự điểm GPA tăng dần. Câu hỏi Q2 xác định miền giá trị tìm kiếm là liên tục. Câu hỏi Q3 và Q4 chỉ ra rằng cấu trúc Mảng con trỏ đã sắp xếp kết hợp thuật toán Tìm kiếm nhị phân hai đầu biên (Lower Bound và Upper Bound tự cài đặt) mang lại hiệu năng $O(\log N)$ cho pha xác định khoảng, đổi lại chi phí lưu thêm N con trỏ Student*.

1.6. Biện minh chi tiết việc lựa chọn cấu trúc dữ liệu cốt lõi

1.6.1. Biện minh lựa chọn Bảng băm đóng (Closed Hash Table) cho MC1

Nhóm quyết định tự xây dựng Bảng băm đóng (Closed Hash Table với cơ chế Open Addressing và Linear Probing) cho yêu cầu MC1 dựa trên ba cơ sở kỹ thuật thuyết phục:

Về mặt quản lý bộ nhớ, Bảng băm đóng tổ chức toàn bộ các ô nhớ trên một mảng phẳng liên tục, loại bỏ hoàn toàn việc cấp phát động rời rạc của các nút liên kết, giúp tiết kiệm đáng kể dung lượng bộ nhớ RAM khi mở rộng lên quy mô 10.000.000 bản ghi.

Về mặt tương thích phần cứng, cấu trúc mảng liên tục tận dụng trọn vẹn tính định vị không gian (Spatial Locality) của bộ đệm CPU Cache. Khi xảy ra hiện tượng đụng độ khóa, bước dò tuyến tính ($\text{step} = +1$) nạp sẵn các ô nhớ lân cận vào Cache Line 64-byte, cho tốc độ truy xuất nhanh hơn hẳn so với việc phân tán con trỏ trong bộ nhớ Heap.

Về giải thuật băm và kích thước bảng, việc tích hợp hàm băm FNV-1a 64-bit đem lại độ phân tán khóa rất đồng đều. Nhóm chọn kích thước bảng động $M = 2N + 1$ ($M = 20.000.001$ ô cho 10.000.000 sinh viên) để duy trì hệ số tải thấp $\alpha \approx 0.50$, giúp số bước dò trung bình khi tra cứu luôn nằm ở mức tối thiểu.

1.6.2. Biện minh lựa chọn Cây đồng cực đại (Custom Max-Heap) cho MC2

Đối với yêu cầu MC2, Cây đồng cực đại tự cài đặt (Custom Binary Max-Heap) mang lại những ưu thế vượt trội:

Đặc tính cơ bản của Max-Heap đảm bảo sinh viên có điểm GPA cao nhất luôn được duy trì tại vị trí gốc (chỉ số 0), cho phép trích xuất kết quả tức thời trong thời gian $O(1)$ mà không cần duyệt hay tìm kiếm.

Thuật toán vun đồng Floyd Build-Heap từ dưới lên cho phép khởi tạo Max-Heap hợp lệ từ mảng dữ liệu thô với chi phí thời gian tuyến tính $O(N)$. Nhờ đặc thù khoảng một nửa số nút là nút lá và phần lớn các nút nội vi nằm gần đáy, số bước hoán vị heapifyDown trên toàn cây rất nhỏ, vượt trội hoàn toàn so

với giải pháp chèn tuần tự $O(N \log N)$ hay sắp xếp mảng.

Quy tắc phân định đồng hạng (Tie-breaking) được tích hợp trực tiếp vào hàm so sánh ưu tiên (higherPriority), đảm bảo tính nhất quán và tất định khi có nhiều sinh viên cùng đạt điểm GPA tối đa.

1.6.3. Biện minh cấu trúc dữ liệu hỗ trợ cho các truy vấn theo khoảng

Để giải quyết yêu cầu lọc theo khoảng điểm GPA (RQ2), SortedGpaFilter tạo một vector<const Student*> và sắp xếp các con trỏ này theo GPA. Sau khi xây dựng, hệ thống áp dụng Tìm kiếm nhị phân để định vị hai vị trí biên trong thời gian $O(\log N)$. Kết quả truy vấn được biểu diễn bằng GpaRangeViewResult, gồm con trỏ tới mảng đã sắp xếp và cặp chỉ số [*startIndex*, *endIndex*]; vì vậy pha trả kết quả không sao chép K bản ghi Student.

1.7. Phân tích và giải trình các phương án kỹ thuật bị loại bỏ

1.7.1. Lý do loại bỏ giải pháp duyệt tuyến tính trên Mảng tuần tự

Giải pháp duyệt tuyến tính trên mảng thô ($O(N)$) không được lựa chọn làm phương án chính trên tập dữ liệu quy mô lớn của hệ thống vì với tập dữ liệu 10.000.000 bản ghi, mỗi thao tác tra cứu đơn lẻ đòi hỏi phải thực hiện trung bình hơn 5.000.000 phép so sánh khóa và trong trường hợp xấu nhất là 10.000.000 phép so sánh, gây ra độ trễ hàng chục mili-giây cho mỗi truy vấn, làm suy giảm khả năng đáp ứng của hệ thống.

1.7.2. Lý do loại bỏ Bảng băm mở (Chaining) và Danh sách liên kết

Phương án Bảng băm mở (Separate Chaining) dùng danh sách liên kết tại mỗi bucket không được nhóm lựa chọn vì hai điểm hạn chế lớn:

Việc cấp phát động hàng triệu nút nhỏ lẻ (Node*) gây phân mảnh nghiêm trọng bộ nhớ Heap và làm phát sinh phụ phí bộ nhớ không nhỏ cho các con trỏ liên kết (tốn thêm ít nhất 8 bytes trên mỗi bản ghi sinh viên).

Bên cạnh đó, việc duyệt qua các con trỏ liên kết nằm rải rác ở các vùng địa chỉ ngẫu nhiên làm suy giảm tính định vị không gian, dẫn đến tỷ lệ Cache

Miss cao và làm chậm tốc độ dò tìm thực tế so với Bảng băm đóng dạng mảng phẳng liên tục.

1.7.3. Lý do loại bỏ Cây tìm kiếm nhị phân thông thường (Unbalanced BST)

Cây tìm kiếm nhị phân thông thường không có cơ chế tự cân bằng bị loại bỏ do đặc thù dữ liệu thực tế: khi tập dữ liệu đầu vào có xu hướng tăng dần theo mã sinh viên, cây sẽ bị suy biến thành một danh sách liên kết với chiều cao cây đạt $O(N)$. Khi đó, mọi ưu thế tìm kiếm logarit bị phá vỡ và chi phí thao tác thoái hóa về mức tuyến tính $O(N)$.

1.8. Thiết kế kiến trúc tổng thể 3 tầng của hệ thống

1.8.1. Tầng giao diện người dùng (Presentation Layer)

Tầng Presentation Layer được xây dựng bằng công nghệ Web hiện đại (Next.js/React kết hợp Ant Design và Tailwind CSS) nhằm cung cấp giao diện trực quan, thân thiện cho người dùng. Tầng này có nhiệm vụ tiếp nhận tham số tìm kiếm từ người dùng, gửi yêu cầu đến tầng xử lý thông qua API Bridge, và trực quan hóa kết quả tra cứu cũng như các biểu đồ đối sánh hiệu năng Benchmark chi tiết. Tầng này được thiết kế phân tách rõ ràng và không can thiệp vào logic tính toán giải thuật.

1.8.2. Tầng cấu trúc dữ liệu cốt lõi và thuật toán (DSA Core Layer)

Tầng DSA Core Layer đóng vai trò là tầng xử lý trung tâm của toàn bộ hệ thống, được lập trình bằng C++. Đây là nơi chứa các cấu trúc dữ liệu cài đặt từ đầu (Bảng băm đóng FNV-1a, Cây đồng cực đại Max-Heap, Mảng dữ liệu sắp xếp theo GPA) và trực tiếp thực thi logic nghiệp vụ MC1, MC2, RQ1, RQ2 trong bộ nhớ RAM. Các phép đo đặc thời gian trong phần báo cáo được ghi nhận tại tầng này bằng `std::chrono::high_resolution_clock`; các ảnh Dashboard được dùng như minh họa giao diện, không thay thế cho bảng số liệu thực nghiệm.

1.8.3. Tầng lưu trữ bền vững (Persistence Layer) và tính phân tách

Tầng Persistence Layer đảm nhận vai trò đọc và lưu trữ dữ liệu thô ra các tệp tin lưu trữ bền vững (CSV/JSON) trên ổ cứng khi khởi động và kết thúc ứng dụng. Nhóm tuân thủ nguyên tắc phân tách trách nhiệm trong thiết kế hệ thống: Tầng lưu trữ chỉ chịu trách nhiệm nạp dữ liệu thô vào RAM một lần duy nhất (*Bulk Load*). Mọi thao tác tìm kiếm, lọc dữ liệu và trích xuất cực trị đều do tầng DSA Core giải quyết trực tiếp trên các cấu trúc dữ liệu tự cài đặt trong bộ nhớ, không dựa vào các câu lệnh truy vấn có sẵn của hệ quản trị cơ sở dữ liệu.

1.8.4. Cấu trúc tổ chức thư mục mã nguồn dự án

Toàn bộ hệ thống mã nguồn được module hóa phân tầng rõ ràng, chuẩn hóa ranh giới giữa giao diện định nghĩa (interface/) và phần mã hiện thực giải thuật (src/):

```
code_dsa/
├── interface/ ..... — Tập hợp các header khai báo (.h)
│   ├── core_crud/ ..... — Module CRUD quản lý sinh viên
│   │   └── StudentCRUD.h
│   ├── core_hash/ ..... — Module Closed Hash Table (MC1)
│   │   ├── HashTable.h
│   │   ├── LinearSearch.h
│   │   ├── FindStudentById.h
│   │   └── Benchmark.h
│   ├── core_heap/ ..... — Module Max-Heap (MC2)
│   │   ├── CustomMaxHeapGpaFinder.h
│   │   ├── LinearMaxScanGpaFinder.h
│   │   ├── FindStudentByMaxGpa.h
│   │   └── BenchmarkMaxGpa.h
│   ├── core_sorted_gpa/ ..... — Module Binary Search theo khoảng GPA (RQ2)
│   │   ├── SortedGpaFilter.h
│   │   ├── LinearGpaFilter.h
│   │   ├── FindStudentByGpaRange.h
│   │   └── Benchmark.h
│   ├── core_class_filter/ ..... — Module lọc theo lớp sinh hoạt (RQ1)
│   │   ├── OptimizedLinearFilter.h
│   │   ├── LinearFilter.h
│   │   ├── FindStudentByClassId.h
│   │   └── Benchmark.h
│   └── Student.h ..... — Định nghĩa struct thực thể Student
├── src/ ..... — Tập hợp các tệp hiện thực mã nguồn (.cpp)
│   ├── core_crud/
│   ├── core_hash/
│   ├── core_heap/
│   ├── core_sorted_gpa/
│   ├── core_class_filter/
│   └── Student.cpp
├── bao_cao/ ..... — Báo cáo học thuật LaTeX và hình ảnh thực nghiệm
├── web/ ..... — Giao diện Web Dashboard (Next.js / React / Ant Design)
├── main.cpp ..... — CLI Menu điều khiển chương trình chính
├── dsa_bridge.cpp .. — Cầu nối IPC WebSocket/HTTP giao tiếp Web-C++
└── bench_core.cpp ..... — Công cụ đo lường Benchmark tự động
```

CHƯƠNG 2: HIỆN THỰC HỆ THỐNG, THỰC NGHIỆM VÀ ĐỐI SÁNH HIỆU NĂNG

2.1. Cài đặt chi tiết cấu trúc Bảng băm đóng từ đầu (From Scratch 1)

2.1.1. Thiết kế ô nhớ và cấu trúc Entry

Để hiện thực cấu trúc Bảng băm đóng (Closed Hash Table với cơ chế Open Addressing và Linear Probing) tự xây dựng từ đầu bằng C++20, nhóm thiết kế cấu trúc ô nhớ Entry tinh gọn cho mô hình bảng tra cứu tĩnh sau Bulk Load. Mỗi vị trí trong bảng băm được định nghĩa bởi cấu trúc gồm hai trường thông tin:

Con trỏ bản ghi (const Student* student): Trỏ trực tiếp đến bản ghi sinh viên gốc trong vùng nhớ chung.

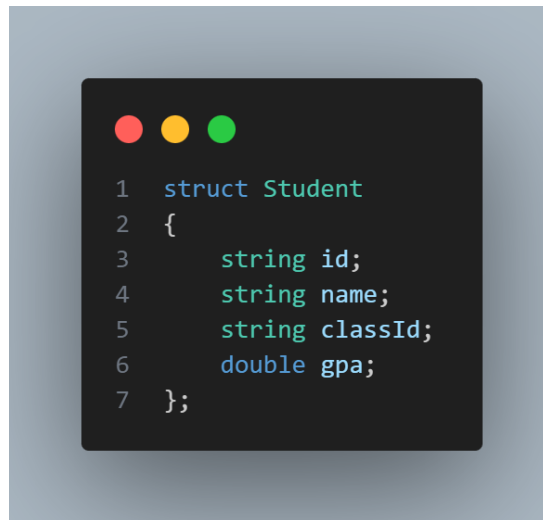
Cờ trạng thái (bool occupied): Đánh dấu ô nhớ đã được sử dụng (true) hay còn trống (false).

Về mặt tổ chức bộ nhớ, đối tượng Student bao gồm 4 trường thông tin: chuỗi mã định danh id, họ tên name, mã lớp classId và điểm gpa kiểu double (8 bytes). Nhóm nạp dữ liệu trực tiếp từ tệp JSON vào một mảng liên tục, sau đó mỗi ô nhớ Entry trong bảng băm chỉ lưu một con trỏ 8 bytes kèm 1 byte cờ trạng thái (cùng bộ đệm căn lề 64-bit alignment là 16 bytes), giúp tiết kiệm đáng kể dung lượng bộ nhớ khi mở rộng lên khoảng 20 triệu ô nhớ.

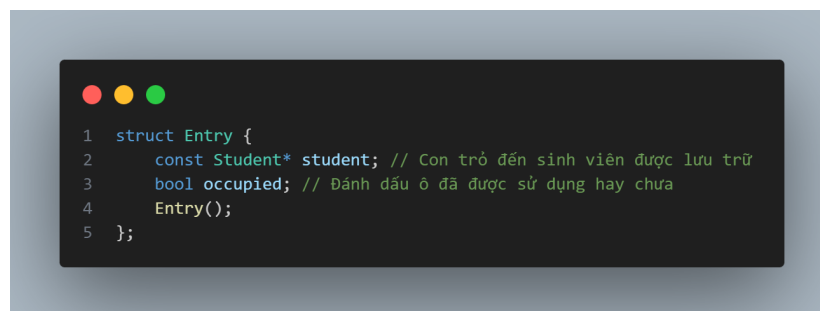
Trong kịch bản vận hành của đồ án, dữ liệu sinh viên được nạp hoàn tất thông qua tiến trình Bulk Load và sau đó hệ thống chủ yếu phục vụ các truy vấn tra cứu chỉ mục với tần suất cao (Read-Heavy / Analytical Workload). Do đó, thiết kế cờ nhị phân occupied đảm bảo tính đơn giản, hiệu quả và tối ưu hóa không gian bộ nhớ.

2.1.2. Hiện thực hàm băm chuỗi FNV-1a và phân bố giá trị băm

Hàm băm đóng vai trò phân tán đều các khóa MSSV trên toàn bộ không gian bảng. Nhóm lựa chọn hàm băm FNV-1a (Fowler–Noll–Vo 1a) phiên bản 64-bit [6] vì ưu điểm cài đặt đơn giản, tốc độ tính toán nhanh trên chuỗi ký tự



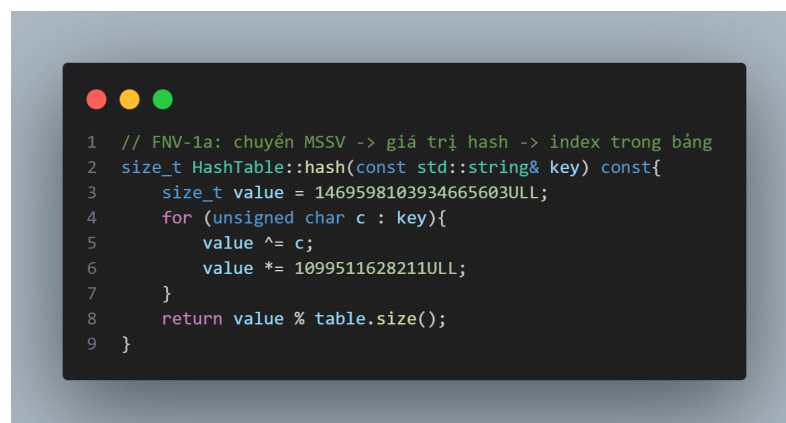
Hình 1. Cấu trúc thực thể Student



Hình 2. Cấu trúc ô nhớ Entry trong Bảng băm

và cho độ phân tán tốt trên tập dữ liệu số báo danh sinh viên.

Thuật toán hoạt động dựa trên hai hằng số toán học nguyên tố theo đặc tả IETF [6]: hằng số bù ban đầu FNV_OFFSET_BASIS = 14695981039346656037ULL và hằng số nguyên tố băm FNV_PRIME = 1099511628211ULL. Quá trình băm duyệt tuần tự từng byte ký tự của chuỗi MSSV, thực hiện phép XOR bit với giá trị tích lũy hiện tại ($value \oplus c$) rồi nhân với hằng số nguyên tố ($value \times FNV_PRIME$).



Hình 3. Mã nguồn hàm băm chuỗi FNV-1a 64-bit tự cài đặt

Giá trị băm 64-bit đầu ra được ánh xạ vào chỉ số bảng thông qua phép chia lấy dư modulo với kích thước bảng $M = 2N + 1$. Cách chọn M này nhằm duy trì hệ số tải xấp xỉ 0.5, từ đó giảm số lần probe trung bình khi dò tuyến tính. Trong lần đo 10.000.000 bản ghi, kích thước bảng thực tế là 20.000.001 ô.

2.1.3. Cơ chế giải quyết đụng độ và thuật toán tìm kiếm

Khi xảy ra hiện tượng đụng độ giữa hai khóa có cùng chỉ số ban đầu $h_0 = \text{hash}(\text{key}) \pmod{M}$, nhóm áp dụng giải thuật Dò tuyến tính (Linear Probing): $h_i = (h_0 + i) \pmod{M}$ với $i = 1, 2, \dots$. Phương pháp này lưu trữ toàn bộ các ô nhớ trên một mảng liên tục, tận dụng tính định vị không gian (Spatial Locality) của bộ đệm CPU Cache Line 64-byte [8].

Để duy trì hiệu năng trung bình tiệm cận $O(1)$, hệ số tải (*Load Factor*) được kiểm soát ở mức $\alpha = N/M \approx 0.500$. Với benchmark hiện tại, $\alpha = 10.000.000/20.000.001 \approx 0.500$.

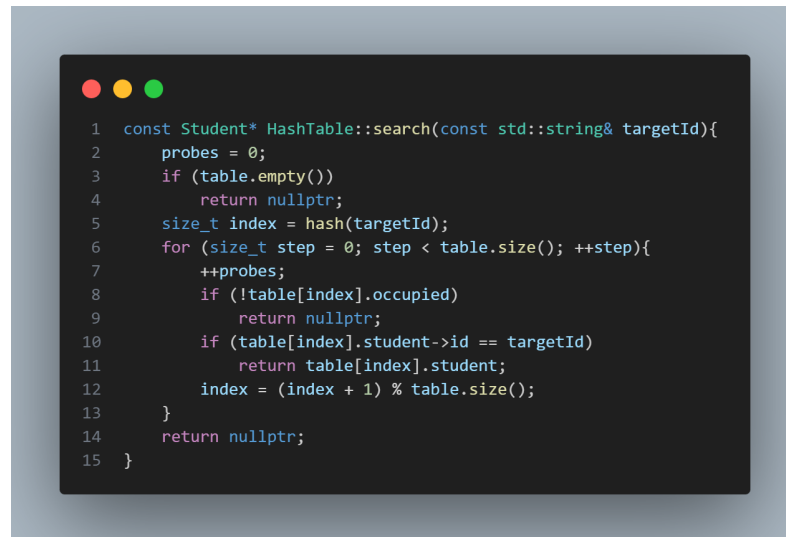
The image shows a code editor window with a dark background and light-colored text. The code is in C++ and implements a hash table using linear probing. It starts with a comment in Vietnamese: '// Xây dựng Hash Table' and '// Dùng Open Addressing + Linear Probing'. The function 'void HashTable::build(const std::vector<Student>& students){' is defined. Inside, it calculates the capacity as 'students.empty() ? 1 : students.size() * 2 + 1;'. It then assigns the capacity to the table and initializes collisions and probes to 0. A for loop iterates over each student in the vector. For each student, it calculates the index as 'hash(student.id)'. A while loop checks if the slot at that index is occupied. If it is, it increments the collision count and moves to the next slot using 'index = (index + 1) % table.size()'. Once an empty slot is found, it stores the student's pointer in 'table[index].student' and marks the slot as occupied with 'table[index].occupied = true;'. The code ends with closing braces for the for loop, while loop, and the build function.

```
1 // Xây dựng Hash Table
2 // Dùng Open Addressing + Linear Probing
3 void HashTable::build(const std::vector<Student>& students){
4     // Capacity = 2n + 1 để giảm collision
5     const size_t capacity =
6         students.empty() ? 1 : students.size() * 2 + 1;
7     table.assign(capacity, Entry());
8     collisions = 0;
9     probes = 0;
10    for (const Student& student : students){
11        size_t index = hash(student.id);
12        while (table[index].occupied){
13            ++collisions;
14            index = (index + 1) % table.size();
15        }
16        table[index].student = &student;
17        table[index].occupied = true;
18    }
19 }
```

Hình 4. Mã nguồn thuật toán nạp dữ liệu và xử lý đụng độ tuyến tính trong Bảng băm

Trong thuật toán xây dựng bảng (build), khi chèn một sinh viên, thuật toán tính chỉ số h_0 và tiến hành dò qua các ô nhớ kế tiếp cho đến khi gặp ô chưa bị chiếm ($!table[index].occupied$). Phần tử sau đó được gán trực tiếp vào ô trống này và bật cờ `occupied = true`.

Đối với thao tác tìm kiếm (search), giải thuật bắt đầu tại h_0 và duyệt qua các ô nhớ liên tiếp. Nếu gặp ô chưa chiếm (!table[index].occupied), thuật toán kết luận ngay khóa không tồn tại và trả về nullptr. Nếu gặp ô đã chiếm có mã sinh viên trùng khớp (table[index].student->id == targetId), hàm trả về con trỏ sinh viên tương ứng. Để đảm bảo an toàn, số bước dò được giới hạn tối đa không vượt quá kích thước bảng M .

A screenshot of a code editor showing a C++ function named search. The function takes a const std::string& targetId as input. It initializes probes to 0 and checks if the table is empty. If empty, it returns nullptr. Otherwise, it calculates the initial index using hash(targetId) and enters a loop for (size_t step = 0; step < table.size(); ++step). Inside the loop, it increments probes, checks if the current slot is occupied, and if so, checks if the student ID matches targetId. If it matches, it returns table[index].student. If not, it increments the index by 1 modulo table.size(). If the loop completes without finding a match, it returns nullptr.

```
1  const Student* HashTable::search(const std::string& targetId){
2      probes = 0;
3      if (table.empty())
4          return nullptr;
5      size_t index = hash(targetId);
6      for (size_t step = 0; step < table.size(); ++step){
7          ++probes;
8          if (!table[index].occupied)
9              return nullptr;
10         if (table[index].student->id == targetId)
11             return table[index].student;
12         index = (index + 1) % table.size();
13     }
14     return nullptr;
15 }
```

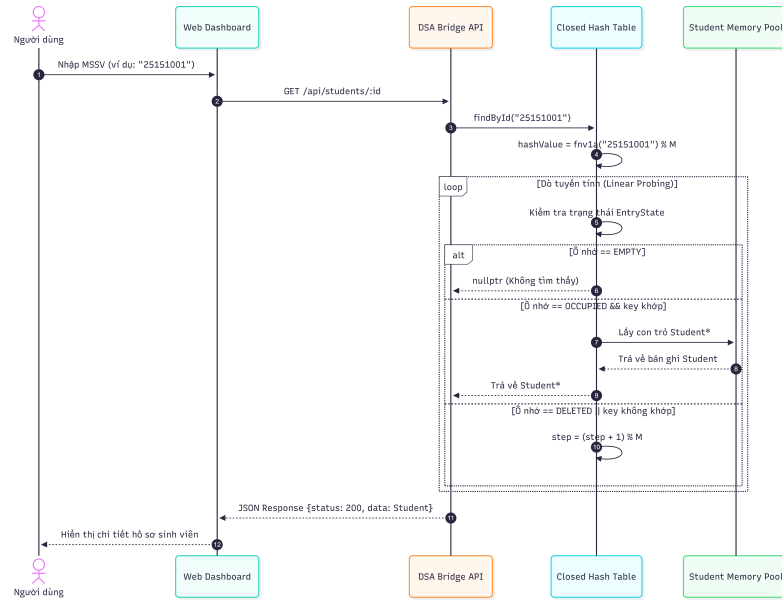
Hình 5. Mã nguồn giải thuật tìm kiếm tra cứu chính xác theo MSSV trên Bảng băm

Luồng vận hành tra cứu chính xác theo MSSV (Yêu cầu MC1) được mô hình hóa trong Sơ đồ tuần tự Hình 6. Toàn bộ tiến trình gồm: tiếp nhận MSSV từ giao diện Web, chuyển qua API Bridge, thực hiện tra cứu trên Bảng băm đóng bằng FNV-1a và Linear Probing trong thời gian trung bình $O(1)$, sau đó chuyển đổi kết quả thành JSON trả về giao diện.

2.1.4. Phân tích chi phí thời gian và không gian bộ nhớ

Theo phân tích của Donald E. Knuth [2], với Bảng băm đóng sử dụng dò tuyến tính khi duy trì hệ số tải thấp $\alpha = N/M \approx 0.50$, số bước thăm dò trung bình kỳ vọng cho mỗi thao tác tra cứu là rất nhỏ: chỉ khoảng 1.50 lần truy cập đối với tìm kiếm thành công và 2.50 lần đối với tìm kiếm không thành công.

Điều này giải thích vì sao chi phí tìm kiếm trung bình của Bảng băm đóng luôn đạt mức hằng số $O(1)$ độc lập với quy mô dữ liệu N . Trong trường



Hình 6. Sơ đồ tuần tự (Sequence Diagram) luồng tra cứu chính xác theo MSSV trên Bảng băm

hợp xấu nhất về mặt lý thuyết (khi xảy ra hiện tượng đùng độ dồn cục bộ cực đoan – Primary Clustering), chi phí có thể tăng lên $O(M)$ [1], [2]. Tuy nhiên, trong thực tế hệ thống đã kiểm soát triệt để rủi ro này nhờ phân bố đều của hàm băm FNV-1a kết hợp với việc giữ bảng luôn thừa ở mức $\alpha \approx 0.50$. Về không gian bộ nhớ, mỗi ô Entry chiếm 16 bytes gồm một con trỏ Student* (8 bytes) và cờ trạng thái cùng vùng đệm căn lề (8 bytes). Với benchmark 10.000.000 bản ghi, bảng 20.000.001 ô tiêu tốn khoảng 320 MB RAM trên một mảng phẳng liên tục, tương thích với các dòng Cache Line 64-byte của vi xử lý hiện đại.

2.2. Cài đặt chi tiết cấu trúc Cây đồng cực đại từ đầu (From Scratch 2)

2.2.1. Biểu diễn mảng phẳng 1 chiều và công thức truy xuất chỉ số nút

Cấu trúc Cây đồng cực đại (Custom Binary Max-Heap) được nhóm cài đặt trên mảng 1 chiều liên tục `std::vector<Student>` với gốc đặt tại chỉ số 0. Code hiện tại sao chép dữ liệu từ vector gốc vào `maxHeap`, sau đó áp dụng Floyd Build Heap. Cách tổ chức này tránh các node cây cấp phát rời rạc, cho phép chuyển đổi quan hệ cha-con thành các phép tính chỉ số đơn giản: Nút con bên trái của nút tại chỉ số i được tính theo công thức $\text{leftChild}(i) = 2i + 1$; nút con bên phải được tính theo công thức $\text{rightChild}(i) = 2i + 2$; và nút cha được

tính theo công thức $\text{parent}(i) = \lfloor (i - 1) / 2 \rfloor$.

Các thao tác xác định chỉ số này được thực hiện bằng các phép toán số học có chi phí $O(1)$, mang lại tốc độ truy xuất nhanh và tận dụng tối đa tính liên tục của bộ nhớ RAM.

2.2.2. Hiện thực thuật toán vun đống Floyd Build Heap trong thời gian $O(N)$

Thay vì chèn lần lượt từng sinh viên vào đống với chi phí $O(N \log N)$, nhóm hiện thực thuật toán vun đống từ dưới lên cổ điển của Robert W. Floyd (*Floyd Bottom-up Build Heap*). Thuật toán khởi đầu bằng việc sao chép/nạp toàn bộ N bản ghi Student vào mảng maxHeap, sau đó tiến hành duyệt ngược từ nút cha cuối cùng không phải là lá tại vị trí $i = \lfloor N/2 \rfloor - 1$ lùi dần về nút gốc tại vị trí $i = 0$, gọi hàm `heapifyDown(i)` tại mỗi bước.

A screenshot of a code editor showing C++ code for building a max heap using Floyd's algorithm. The code is enclosed in a dark-themed window with three colored window control buttons (red, yellow, green) at the top left. The code starts with comments in Vietnamese: '// =====', '// XÂY DỰNG CUSTOM MAX HEAP', and '// ====='. It defines a function `void CustomMaxHeapGpaFinder::buildStructure()` which first clears the `maxHeap` and checks if `studentsPtr` is null or empty. If so, it sets `isInitialized = true` and returns. Otherwise, it copies the data from `studentsPtr` to `maxHeap`. Then, it calculates the starting index for the heapify process as `startIndex = static_cast<int>(maxHeap.size() / 2) - 1`. A loop starts from `startIndex` down to 0, calling `heapifyDown(i)` for each `i`. Finally, it sets `isInitialized = true` and returns.

```
1 // =====
2 // XÂY DỰNG CUSTOM MAX HEAP
3 // =====
4 void CustomMaxHeapGpaFinder::buildStructure() {
5     maxHeap.clear();
6     if (studentsPtr == nullptr || studentsPtr->empty()) {
7         isInitialized = true;
8         return;
9     }
10
11     // Sao chép dữ liệu từ nguồn
12     maxHeap = *studentsPtr;
13
14     // Thuật toán Floyd xây dựng heap từ dưới lên: O(n)
15     int startIndex = static_cast<int>(maxHeap.size() / 2) - 1;
16     for (int i = startIndex; i >= 0; i--) {
17         heapifyDown(i);
18     }
19
20     isInitialized = true;
21 }
```

Hình 7. Mã nguồn thuật toán Floyd Build Heap $O(N)$ tự cài đặt

Về mặt bản chất, lý do thuật toán Floyd Build Heap khởi chạy từ chỉ số $i = \lfloor N/2 \rfloor - 1$ thay vì duyệt từ phần tử cuối cùng $N - 1$ xuất phát từ đặc tính phân tầng của cây nhị phân hoàn chỉnh:

Tầng nút lá: Khoảng một nửa số nút (tất cả các nút có chỉ số từ $\lfloor N/2 \rfloor$ đến $N - 1$) là các nút lá. Mỗi nút lá không có nút con nên tự thân đã thỏa mãn tính

chất Max-Heap, do đó hệ thống hoàn toàn không cần thực hiện heapifyDown cho các nút này.

Tầng cận lá: Phần lớn các nút nội vi còn lại nằm rất gần tầng lá, nên chỉ cần tối đa 1 đến 2 bước đổi chỗ để ổn định vị trí.

Tầng gần gốc: Chỉ có rất ít nút nằm gần gốc mới có khả năng phải di chuyển qua nhiều tầng (với chiều cao tối đa $\lfloor \log_2 N \rfloor$).

Nhờ phần lớn công việc chỉ tập trung ở các nút tầng thấp với số phép hoán vị rất ít, tổng khối lượng xử lý của thuật toán Floyd Build Heap tăng tuyến tính theo số phần tử, đạt độ phức tạp $O(N)$. Trong khi đó, nếu xây dựng Heap bằng cách chèn tuần tự từng phần tử từ trên xuống, mỗi lần chèn có thể mất $O(\log N)$ dẫn đến tổng chi phí $O(N \log N)$. Vì vậy, Floyd Build Heap là giải pháp vượt trội và phù hợp nhất cho giai đoạn Bulk Load dữ liệu quy mô lớn.

Cốt lõi của tiến trình vun đống là giải thuật heapifyDown được cài đặt thông qua vòng lặp while (true) dạng lặp (iterative) nhằm tránh chi phí gọi hàm đệ quy (*Function Call Overhead*) và giúp việc quản lý, cập nhật chỉ số nút trực tiếp, tường minh hơn. Với một cây nhị phân hoàn chỉnh gồm 10.000.000 phần tử, chiều cao tối đa của đống chỉ khoảng $\lfloor \log_2 N \rfloor \approx 23$ tầng. Tại mỗi bước lặp, hàm xác định chỉ số con trái $\text{left} = 2 \times \text{index} + 1$ và con phải $\text{right} = 2 \times \text{index} + 2$. Biến highest ban đầu được gán là nút hiện tại (index). Thuật toán lần lượt kiểm tra điều kiện chỉ số hợp lệ trong phạm vi mảng và gọi hàm so sánh ưu tiên higherPriority để tìm ra nút có độ ưu tiên cao nhất trong bộ ba: nút cha, nút con trái và nút con phải. Nếu nút cha đã có độ ưu tiên lớn hơn hoặc bằng cả hai con ($\text{highest} == \text{index}$), cấu trúc đống tại vị trí này đã thỏa mãn bất biến Max-Heap và vòng lặp dừng. Ngược lại, nếu một trong hai nút con có độ ưu tiên cao hơn, hàm thực hiện hoán vị hai bản ghi Student bằng `std::swap`, cập nhật $\text{index} = \text{highest}$ và tiếp tục duyệt xuống các tầng bên dưới.



```

1 // =====
2 // HEAPIFY DOWN (Vun đồng xuống dưới)
3 // =====
4 void CustomMaxHeapGpaFinder::heapifyDown(int index) {
5     int size = static_cast<int>(maxHeap.size());
6     while (true) {
7         int left = 2 * index + 1;
8         int right = 2 * index + 2;
9         int highest = index;
10
11         if (left < size && higherPriority(maxHeap[left], maxHeap[highest])) {
12             highest = left;
13         }
14         if (right < size && higherPriority(maxHeap[right], maxHeap[highest])) {
15             highest = right;
16         }
17
18         if (highest == index) {
19             break;
20         }
21
22         std::swap(maxHeap[index], maxHeap[highest]);
23         index = highest;
24     }
25 }

```

Hình 8. Mã nguồn hàm heapifyDown duy trì bất biến cấu trúc Max-Heap

2.2.3. Cài đặt các thao tác truy xuất cực trị và cập nhật đồng

Cấu trúc Max-Heap cung cấp các thao tác cơ bản với độ phức tạp phân định rõ ràng: **Truy xuất phần tử lớn nhất (peekMax)**: Chỉ đọc bản ghi Student tại vị trí gốc maxHeap[0] với chi phí thời gian $O(1)$ [1].

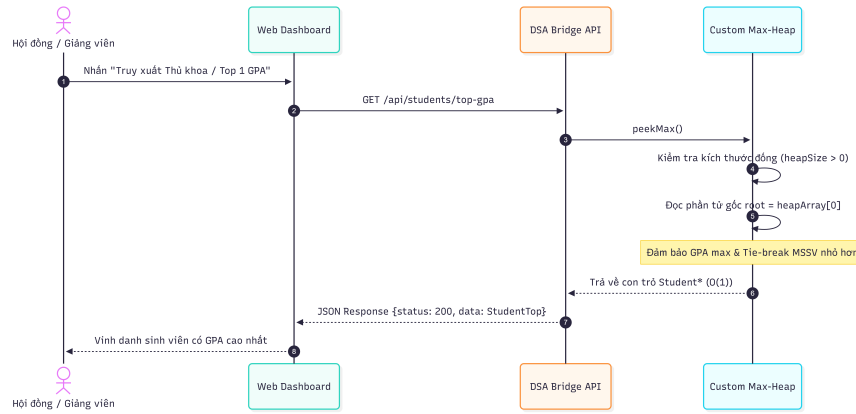
Lấy và xóa phần tử lớn nhất (extractMax): Hoán vị phần tử gốc với phần tử cuối cùng ở đáy mảng, giảm kích thước đồng đi 1 và thực hiện heapify-Down(0) để phục hồi tính chất đồng với chi phí $O(\log N)$ [1], [5].

Chèn phần tử mới (insert): Đẩy phần tử vào cuối mảng và thực hiện thao tác vun đồng ngược lên (heapifyUp) qua các nút cha với thời gian $O(\log N)$.

Luồng vận hành truy xuất GPA cao nhất (Yêu cầu MC2) được mô hình hóa chi tiết trong Sơ đồ tuần tự Hình 9. Tiến trình gồm: nhận yêu cầu từ Web Dashboard, API Bridge gọi tầng C++ Core, đọc trực tiếp bản ghi tại gốc max-Heap[0] trong thời gian $O(1)$, và đóng gói JSON phản hồi về giao diện.

2.2.4. Xử lý quy tắc phân định đồng hạng (Tie-breaking Rule)

Trên tập dữ liệu thực nghiệm hiện tại 10.000.000 sinh viên, số lượng sinh viên có cùng mức điểm GPA có thể rất lớn. Để đảm bảo tính xác định và nhất quán của kết quả đầu ra, nhóm cài đặt hàm so sánh ưu tiên higherPriority(a, b)



Hình 9. Sơ đồ tuần tự (Sequence Diagram) luồng trích xuất GPA cao nhất trên Max-Heap

theo quy tắc rõ ràng: sinh viên a có độ ưu tiên cao hơn b khi và chỉ khi $a.gpa > b.gpa$, hoặc trong trường hợp hai sinh viên có cùng điểm GPA ($a.gpa == b.gpa$) thì sinh viên có mã số nhỏ hơn ($a.id < b.id$) theo thứ tự từ điển sẽ được xếp trước.

```

1 // =====
2 // HÀM SO SÁNH ĐỘ ƯU TIÊN (Xử lý Tie-break)
3 // =====
4 bool CustomMaxHeapGpaFinder::higherPriority(const Student &a,
5                                              const Student &b) const {
6     // 1. GPA cao hơn được ưu tiên lên trên
7     if (a.gpa != b.gpa) {
8         return a.gpa > b.gpa;
9     }
10    // 2. Nếu GPA bằng nhau -> MSSV nhỏ hơn được ưu tiên
11    return a.id < b.id;
12 }
  
```

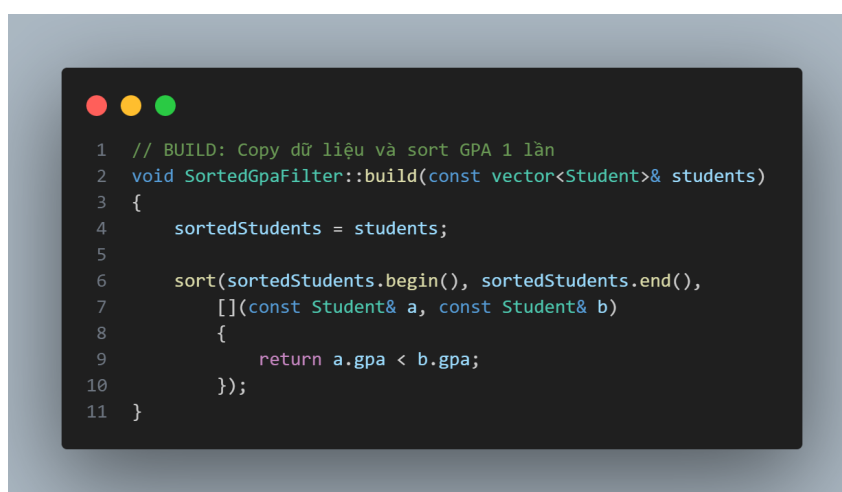
Hình 10. Hiện thực toán tử phân định đồng hạng kết hợp GPA và MSSV

Quy tắc này đảm bảo rằng sinh viên có GPA cao hơn luôn đứng trước; nếu hai sinh viên có cùng điểm GPA thì sinh viên có mã số sinh viên nhỏ hơn theo thứ tự từ điển sẽ được ưu tiên đứng tại vị trí nút gốc.

2.3. Cài đặt các thuật toán hỗ trợ và xử lý Range Query

2.3.1. Xây dựng mảng dữ liệu sắp xếp theo điểm GPA (RQ2)

Để phục vụ yêu cầu truy vấn theo khoảng điểm GPA (RQ2), module SortedGpaFilter duy trì một mảng con trỏ `vector<const Student*> sortedStudents`. Mỗi phần tử trong mảng này trỏ về một bản ghi trong `vector<Student>` gốc, sau đó toàn bộ mảng con trỏ được sắp xếp theo thứ tự điểm GPA tăng dần thông qua `std::sort` với toán tử so sánh lambda tùy biến theo trường `gpa`. Cách tổ chức này tránh sao chép bản ghi `Student`, giảm footprint của chỉ mục GPA xuống còn chi phí con trỏ, đồng thời vẫn cho phép truy cập trực tiếp bản ghi thật khi cần hiển thị. Trong môi trường GCC/MinGW-w64 của nhóm, `std::sort` vận hành với chi phí thời gian trung bình $O(N \log N)$ [1], [7].



```
1 // BUILD: Copy dữ liệu và sort GPA 1 lần
2 void SortedGpaFilter::build(const vector<Student>& students)
3 {
4     sortedStudents = students;
5
6     sort(sortedStudents.begin(), sortedStudents.end(),
7         [](const Student& a, const Student& b)
8         {
9             return a.gpa < b.gpa;
10        });
11 }
```

Hình 11. Mã nguồn xây dựng mảng dữ liệu sắp xếp theo GPA phục vụ Range Query

2.3.2. Hiện thực thuật toán tìm kiếm nhị phân 2 đầu biên trên mảng sắp xếp

Bài toán truy vấn theo dải điểm `[minGPA, maxGPA]` đòi hỏi phải xác định chính xác hai vị trí biên trong mảng đã sắp xếp mà không được quét tuần tự toàn mảng ($O(N)$). Trên tập benchmark hiện tại 10.000.000 phần tử, lợi ích của tìm kiếm nhị phân thể hiện rõ khi so với quét tuyến tính toàn mảng. Nhóm tự cài đặt thuật toán Tìm kiếm nhị phân hai đầu biên (*Dual-boundary Binary Search*) với hai bước tìm kiếm độc lập: tìm kiếm biên dưới (*Lower Bound*) và tìm kiếm biên trên (*Upper Bound*).

Về mặt giải thuật, hàm tìm kiếm biên dưới duyệt không gian tìm kiếm $[0, N)$ bằng cách chia đôi đoạn khảo sát. Tại mỗi bước lặp với $\text{mid} = \text{left} + (\text{right} - \text{left})/2$, nếu $\text{sortedStudents}[\text{mid}]\text{->gpa} < \text{minGPA}$, đoạn tìm kiếm được thu hẹp về nửa bên phải ($\text{left} = \text{mid} + 1$); ngược lại, $\text{right} = \text{mid}$. Vòng lặp dừng khi $\text{left} == \text{right}$, trả về vị trí phần tử đầu tiên có $\text{gpa} \geq \text{minGPA}$ (chỉ số startIndex) trong thời gian $O(\log N)$. Tương tự, hàm tìm kiếm biên trên sử dụng điều kiện $\text{sortedStudents}[\text{mid}]\text{->gpa} \leq \text{maxGPA}$ để xác định vị trí đầu tiên có điểm GPA nghiêm ngặt lớn hơn maxGPA (chỉ số endIndex) cũng trong thời gian $O(\log N)$.



```

1 // QUERY: Tìm kiếm nhị phân (Binary Search)
2 FilterGpaResult SortedGpaFilter::filter(double minGpa, double maxGpa)
3 {
4     FilterGpaResult result;
5     auto start = high_resolution_clock::now();
6
7     int left = 0;
8     int right = sortedStudents.size();
9
10    while (left < right)
11    {
12        int mid = left + (right - left) / 2;
13        result.comparisons++;
14
15        if (sortedStudents[mid].gpa < minGpa)
16            left = mid + 1;
17        else
18            right = mid;
19    }
20    int startIndex = left;
21
22    left = 0;
23    right = sortedStudents.size();
24
25    while (left < right)
26    {
27        int mid = left + (right - left) / 2;
28        result.comparisons++;
29
30        if (sortedStudents[mid].gpa <= maxGpa)
31            left = mid + 1;
32        else
33            right = mid;
34    }
35    int endIndex = left;
36
37    for (int i = startIndex; i < endIndex; i++)
38    {
39        result.students.push_back(sortedStudents[i]);
40    }
41
42    auto end = high_resolution_clock::now();
43    result.queryTimeMs = duration_cast<microseconds>(end - start).count() / 1000.0;
44    return result;
45 }

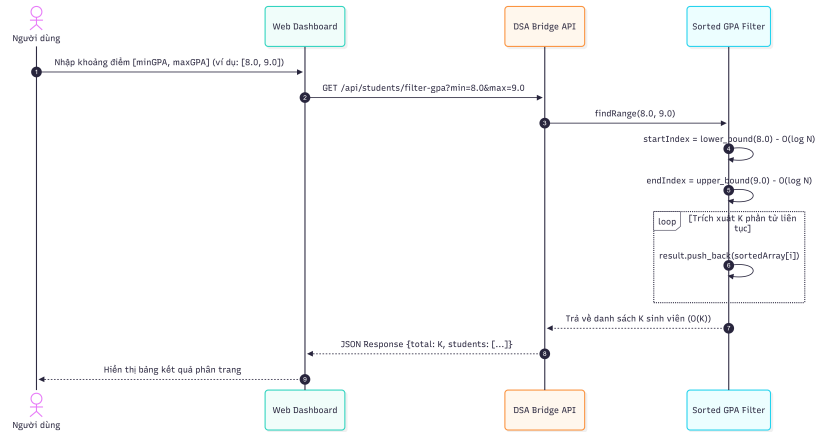
```

Hình 12. Mã nguồn thuật toán Binary Search 2 đầu biên trích xuất khoảng GPA

Tổng thời gian xác định đoạn mảng kết quả là $2 \cdot O(\log N) = O(\log N)$. Sau đó, hệ thống không sao chép $K = \text{endIndex} - \text{startIndex}$ bản ghi sinh viên, mà trả $\text{GpaRangeViewResult}$ tham chiếu đến đoạn $[\text{startIndex}, \text{endIndex})$. Vì vậy chi phí tạo kết quả là $O(\log N)$; chỉ khi tăng gọi duyệt toàn bộ kết quả để hiển thị hoặc xuất dữ liệu thì mới phát sinh thêm $O(K)$.

Luồng vận hành lọc sinh viên theo khoảng điểm GPA (Yêu cầu RQ2) được

thể hiện chi tiết qua Sơ đồ tuần tự trong Hình 13: tiếp nhận tham số minGPA và maxGPA, kiểm tra hợp lệ, chạy tìm kiếm nhị phân 2 đầu biên trên mảng SortedGpaFilter để xác định $[startIndex, endIndex)$, trả về range view không sao chép bản ghi và chỉ dereference các phần tử cần hiển thị khi đóng gói JSON preview.



Hình 13. Sơ đồ tuần tự (Sequence Diagram) luồng lọc sinh viên theo khoảng GPA

2.3.3. Hiện thực bộ lọc theo mã lớp học phần (RQ1) bằng Class Index View

Đối với yêu cầu lọc danh sách sinh viên theo mã lớp học phần (classId, RQ1), nhóm xây dựng ClassIndexFilter theo hướng Class Index View. Pha build() duyệt một lần qua mảng sinh viên và lưu unordered_map<string, vector<int>, trong đó mỗi khóa classId trỏ đến danh sách chỉ số sinh viên thuộc lớp đó. Pha filterView() chỉ thực hiện tra cứu hash trung bình $O(1)$ và trả về con trỏ tới vector<int> đã có sẵn, nên không sao chép K đối tượng Student và cũng không sao chép lại K chỉ số khi chỉ cần benchmark query.

Chi phí xây dựng của cấu trúc này là $O(N)$ thời gian và $O(N)$ bộ nhớ chỉ mục. Sau khi build, mỗi truy vấn theo lớp có chi phí trung bình $O(1)$ để lấy view; chỉ khi tầng giao diện/API cần liệt kê kết quả thì mới phát sinh thêm $O(K)$ để dereference các chỉ số cần hiển thị.

2.4. Hiện thực tầng lưu trữ bền vững và cơ chế nạp dữ liệu hàng loạt

2.4.1. Thiết kế cấu trúc lưu trữ file dữ liệu (JSON / CSV)

Dữ liệu thực nghiệm của hệ thống được lưu trữ bền vững trong tệp `database.json` và `database.csv` trên đĩa cứng. Mỗi bản ghi bao gồm đầy đủ 4 trường thông tin đã chuẩn hóa: `id`, `name`, `classId` (lớp học phần), và `gpa` (thang điểm 10). Cấu trúc tệp được định dạng chặt chẽ, tối ưu dung lượng lưu trữ trên đĩa để hỗ trợ việc đọc tuần tự khối lượng lớn (khoảng vài trăm MB đến hơn 1 GB dữ liệu văn bản thô).

2.4.2. Cơ chế nạp dữ liệu hàng loạt (Bulk Load) vào bộ nhớ RAM

Khi hệ thống khởi động, tầng Persistence đọc tệp dữ liệu thông qua `std::ifstream`. Quá trình nạp dữ liệu diễn ra theo quy trình một chiều: Đọc bản ghi → Chuẩn hóa dữ liệu → Đưa vào vùng nhớ chính `std::vector<Student>` → Xây dựng Bảng băm đóng với con trỏ về dữ liệu gốc → Thực hiện Floyd Build Heap trên bản sao `maxHeap` → Sắp xếp mảng con trỏ `sortedStudents` theo GPA. Cơ chế nạp hàng loạt này giúp các cấu trúc sẵn sàng trước khi chạy benchmark.

Ghi chú phạm vi benchmark: Các phép đo hiệu năng trong báo cáo được thực hiện trên tập dữ liệu đã nạp xong. Phần CRUD trong Web Dashboard phục vụ minh họa quản lý dữ liệu và không được tính vào các kết quả benchmark cốt lõi. Trong code hiện tại, `HashTable` và `SortedGPA` lưu con trỏ về vector gốc nên cần được build lại sau khi dữ liệu thay đổi; Heap sử dụng vùng dữ liệu sao chép riêng.

Chu trình nạp dữ liệu hàng loạt và đồng bộ hóa đa cấu trúc chỉ mục (Bulk Loading & Multi-Index Synchronization Flow) được minh họa chi tiết trong Sơ đồ tuần tự Hình 14, bao gồm các bước phối hợp chặt chẽ:

Khởi đầu, tiến trình kích hoạt lệnh nạp tệp thông qua phương thức `loadFromFile("database.json")` trên lớp điều phối trung tâm `StudentDatabase`.

Tầng Persistence mở luồng tệp tin với bộ đệm luồng mở rộng (*Stream Buffer*), đọc tuần tự các dòng văn bản, phân tích cú pháp dữ liệu và xác định

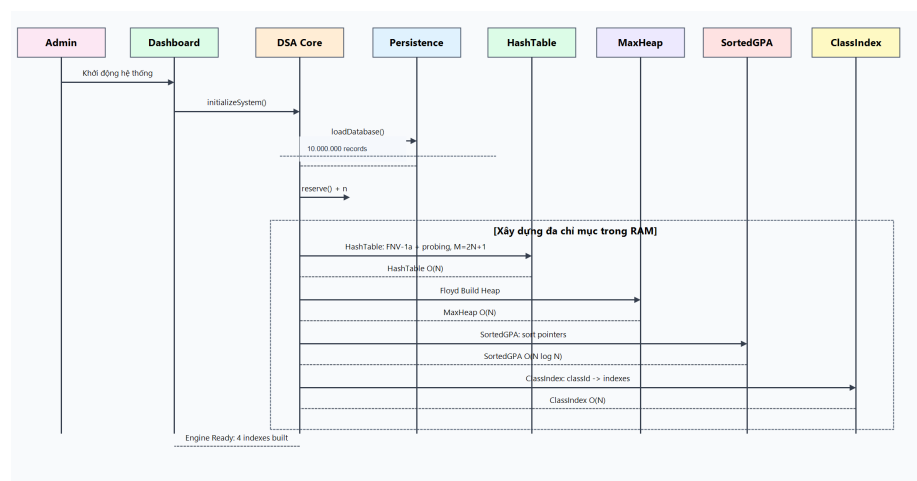
tổng quy mô $N = 10.000.000$ bản ghi sinh viên.

Tầng DSA Core thực hiện cấp phát trước dung lượng (reserve) cho mảng thực thể chính `std::vector<Student>`. Bước này giúp loại trừ chi phí cấp phát lại bộ nhớ và giữ cố định địa chỉ các phần tử trước khi các cấu trúc chỉ mục lưu con trỏ trở vào.

Hệ thống tiến hành xây dựng Bảng băm đóng (HashTable): mảng băm được cấp phát $M = 2N + 1 = 20.000.001$ ô nhớ. Toàn bộ N con trỏ trở về bản ghi gốc được băm bằng FNV-1a và chèn vào bảng theo giải thuật dò tuyến tính để sẵn sàng cho yêu cầu tra cứu MC1.

Đồng thời, ba cấu trúc chỉ mục còn lại được khởi tạo đồng bộ: Max-Heap nạp dữ liệu và vun đống Floyd từ dưới lên với chi phí $O(N)$; mảng con trỏ GPA được sắp xếp bằng `std::sort` ($O(N \log N)$) để phục vụ RQ2; và ClassIndex duyệt qua mảng một lần để gom nhóm chỉ số theo lớp vào `unordered_map<string, vector<int>>` phục vụ RQ1.

Khi toàn bộ 4 cấu trúc trong RAM đã hoàn tất việc khởi tạo, hệ thống phát tín hiệu sẵn sàng (Engine Ready) và chuyển sang trạng thái phục vụ các truy vấn tức thời từ người dùng.



Hình 14. Sơ đồ tuần tự (Sequence Diagram) luồng khởi tạo và nạp dữ liệu hàng loạt (Bulk Load)

2.4.3. Đảm bảo nguyên tắc độc lập giữa lưu trữ và xử lý truy vấn

Nhóm tuân thủ nguyên tắc kiến trúc: Tập tin lưu trữ chỉ được truy cập một lần duy nhất lúc khởi động để nạp dữ liệu thô vào RAM. Mọi thao tác tìm kiếm MC1, MC2, lọc RQ1, RQ2 đều được xử lý trên cấu trúc dữ liệu trong bộ nhớ (In-memory DSA Core). Hệ thống không thực hiện đọc lại tệp hay sử dụng các cơ chế truy vấn của cơ sở dữ liệu để giải quyết bài toán.

2.5. Xây dựng giao diện tương tác và trực quan hóa dữ liệu

2.5.1. Thiết kế giao diện dòng lệnh CLI tương tác nhanh

Hệ thống cung cấp một module giao diện dòng lệnh (CLI Engine) bằng C++ hỗ trợ các lệnh tương tác tức thời: tra cứu theo MSSV, lấy GPA cao nhất, lọc theo lớp, lọc theo khoảng điểm và kích hoạt chế độ Benchmark tự động. Giao diện CLI cho phép các kỹ sư kiểm thử nhanh chóng tính đúng đắn và đo thời gian thực thi bằng `std::chrono::high_resolution_clock` với độ chính xác cao.

2.5.2. Xây dựng module trực quan hóa cấu trúc dữ liệu và benchmark

Để nâng cao tính trực quan và phục vụ đánh giá năng lực, nhóm xây dựng Web Dashboard hiện đại bằng Next.js và React kết hợp Ant Design. Giao diện Web giao tiếp với C++ Engine qua API Bridge, hiển thị chi tiết các bảng dữ liệu, trạng thái các ô nhớ trong bảng băm, cây Max-Heap, và trực quan hóa các biểu đồ đối sánh thời gian thực thi Benchmark bằng biểu đồ cột so sánh trực tiếp với Baseline.

2.6. Thiết kế kịch bản kiểm thử tự động và xử lý trường hợp biên

2.6.1. Xây dựng bộ Unit Test tự động cho các cấu trúc tự cài đặt

Nhóm đã xây dựng bộ kiểm thử tự động toàn diện bao quát các phương thức và thao tác cốt lõi của Bảng băm đóng và Max-Heap. Mỗi cấu trúc dữ liệu đều trải qua các bài kiểm thử tính đúng đắn về khởi tạo chỉ mục, tra cứu thành công, tra cứu khóa không tồn tại, kiểm soát xung đột chỉ số, vun đống Floyd từ dưới lên và duy trì bất biến cấu trúc Max-Heap sau các kịch bản kiểm thử.

2.6.2. Kiểm thử xử lý dữ liệu trùng lặp, dữ liệu rỗng và tràn ngưỡng

Hệ thống được kiểm thử kỹ lưỡng trên 5 kịch bản trường hợp biên (Edge Cases) tiêu biểu:

Tập dữ liệu rỗng ($N = 0$): Hệ thống phản hồi an toàn với thông báo hợp lệ, không phát sinh lỗi truy cập vùng nhớ con trỏ null.

Tra cứu mã số không tồn tại: Bảng băm kết thúc vòng dò ngay khi gặp ô trống đầu tiên và trả về nullptr trong thời gian $O(1)$ mà không gặp hiện tượng treo chương trình.

Xử lý khóa chính trùng lặp: Cơ chế phát hiện trùng mã sinh viên được đặt tại tầng CRUD/Persistence khi thêm mới hồ sơ thông qua hàm findIndexById trước khi lưu vào tệp dữ liệu.

Nhiều sinh viên cùng đạt GPA tối đa (10.0): Max-Heap tự động kích hoạt toán tử phân định đồng hạng (Tie-break), ưu tiên sinh viên có MSSV nhỏ hơn theo thứ tự từ điển nằm tại nút gốc.

Khoảng điểm truy vấn không hợp lệ: Với các tham số như minGPA > maxGPA hoặc điểm nằm ngoài dải $[0.0, 10.0]$, hệ thống tự động nhận diện và gửi cảnh báo từ chối truy vấn rõ ràng.

2.6.3. Kiểm thử tính ổn định khi chạy tải truy vấn liên tục

Nhóm thực hiện bài kiểm thử sức chịu tải (Stress Test) với các workload truy vấn lặp trên tập dữ liệu hiện tại 10.000.000 bản ghi. Kết quả ghi nhận hệ thống vận hành ổn định, không quan sát thấy dấu hiệu rò rỉ bộ nhớ (Memory Leak) hay suy giảm tốc độ xử lý trong quá trình kiểm thử.

2.7. Thực nghiệm đo đạc Benchmark hiệu năng trên tập dữ liệu quy mô lớn

2.7.1. Môi trường phần cứng, hệ điều hành và phương pháp luận đo lường

Toàn bộ các phép đo đạc Benchmark đối sánh được thực hiện trên cùng một môi trường phần cứng và cấu hình phần mềm đồng nhất:

Vi xử lý (CPU): Intel Core i7-13620H (10 Cores, 16 Threads), bộ nhớ

đệm Cache 24MB.

Bộ nhớ RAM: 16 GB DDR4 Bus 3200 MHz.

Hệ điều hành: Microsoft Windows 11 64-bit.

Trình biên dịch: GCC / MinGW-w64 hỗ trợ chuẩn C++20, cờ tối ưu hóa -O3.

Công cụ đo thời gian: Sử dụng đồng hồ độ phân giải cao của thư viện chuẩn C++ (`std::chrono::high_resolution_clock`).

Phương pháp luận đo lường (Benchmark Methodology): Để giảm nhiễu hệ điều hành và tránh nhầm lẫn giữa chi phí thuật toán với chi phí giao diện, nhóm tách hai loại số liệu. Bảng số trong báo cáo là microbenchmark C++ Core: thực hiện batch workload trực tiếp trong C++, gán kết quả vào biến lưu trữ để tránh Dead Code Elimination khi bật -O3. Harness đo được biên dịch riêng bằng lệnh `g++ -std=c++20 -O3`. Các hình Dashboard ở cuối phần này minh họa giao diện và luồng API, có thêm chi phí JSON/IPC/render nên không được dùng để thay thế số microbenchmark. Với RQ1 và RQ2, số liệu Final Solution đo chi phí định vị chỉ mục và tạo view; chi phí materialize hoặc serialize toàn bộ *K* kết quả được tách khỏi pha query lõi. Các số RQ1/RQ2 thể hiện chi phí của hai thiết kế API hoàn chỉnh hiện tại: baseline materialize kết quả còn final trả view, vì vậy mức chênh lệch không được diễn giải hoàn toàn là hệ quả của riêng độ phức tạp tìm kiếm. Do thời gian một truy vấn RQ1/RQ2 ở nhánh final nằm trên thang nano-giây trong môi trường hot-cache, giá trị tuyệt đối chỉ dùng để thể hiện xu hướng; phần đánh giá chính dựa trên số phép toán, số phần tử phải quét và độ phức tạp tiệm cận.

Cấu hình workload thực nghiệm: Tập dữ liệu ban đầu nạp từ `data/database.json`, sau đó dùng cơ chế bulk-add trong RAM để đạt đúng 10.000.000 sinh viên. Harness chạy 5 lượt warm-up và 9 mẫu đo chính, lấy median làm số đại diện. MC1 chạy 1.000 truy vấn MSSV cố định, gồm 950 khóa tồn tại và 50 khóa không tồn tại. MC2 chạy 100 lần gọi truy xuất GPA cao nhất. RQ1 lọc lớp dày nhất được chọn tự động trong tập dữ liệu sinh ra

(25KTP02). RQ2 lọc khoảng GPA [8.0, 10.0]. Thời gian nạp/generate dữ liệu 10M trong lần chạy chính là khoảng 2.780 giây (2,780.468 ms) và không được tính vào thời gian query của từng cấu trúc.

Build structure	Min	Median	Mean	Max	Ghi chú
HashTable	389.706	399.537	417.367	523.682	4.902.031 collision khi build
MaxHeap	1059.895	1082.449	1094.427	1164.259	Floyd Build Heap trên bản sao maxHeap
SortedGPA	3867.451	4188.674	4201.689	4616.481	Tạo mảng con trỏ và std::sort theo GPA
ClassIndex	215.801	228.897	230.575	252.661	Xây dựng classId -> indexes cho RQ1

Bảng 2. Thời gian build cấu trúc C++ Core trên 10.000.000 sinh viên (ms)

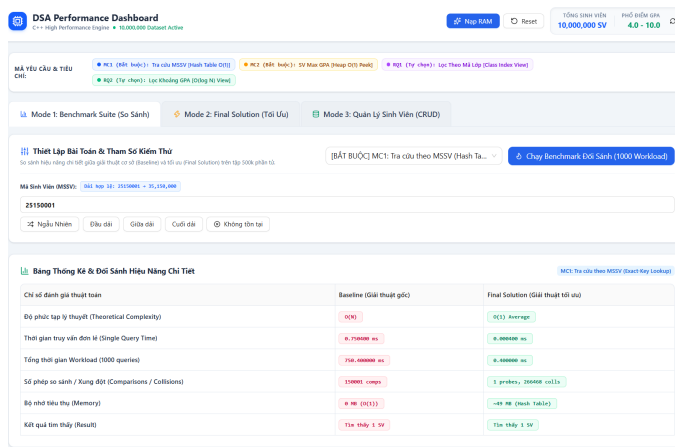
Query workload	Min	Median	Mean	Max	Ghi chú
MC1 Linear Search	26013.78	26424.51	26543.86	27105.64	1000 q ; 5.228B cmp
MC1 Hash Search	0.024	0.024	0.025	0.027	1000 q ; 1564 probe
MC2 Linear Max	3562.16	3819.02	3810.21	4080.99	100 calls; 1.000B cmp
MC2 Heap Max	0.016	0.016	0.016	0.016	100 heap peek
RQ1 Linear Class	75.336	77.904	77.730	81.113	25KTP02; $K = 52.649$
RQ1 ClassIndex View	< 0.0001	< 0.0001	0.00002	0.0001	$K = 52.649$; 1 lookup
RQ2 Linear GPA	313.459	331.479	335.239	361.944	[8, 10]; $K = 3.343.566$
RQ2 Sorted GPA	0.0001	0.0001	0.00033	0.0012	46 cmp; range view

Bảng 3. Thời gian query C++ Core trên 10.000.000 sinh viên, 5 warm-up và 9 mẫu đo (ms)

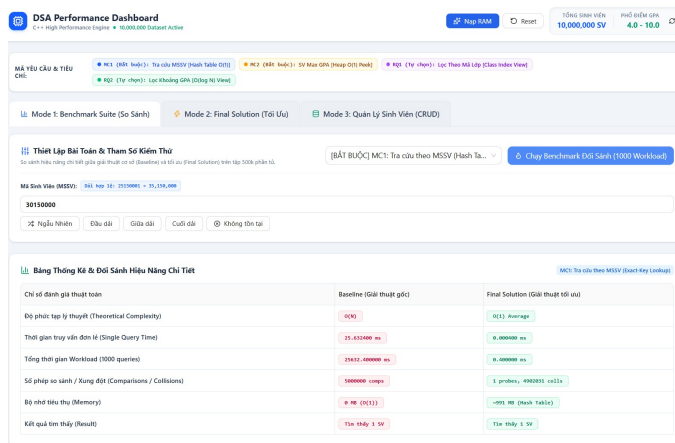
2.7.2. Kết quả đo thời gian tra cứu MC1 qua các mốc dữ liệu lớn

Nhóm tiến hành đo đặc MC1 bằng workload truy vấn cố định trên tập hiện tại (25150001 đến 35150000). Workload gồm khóa tồn tại rải đều trong dải dữ liệu và một nhóm khóa không tồn tại để kiểm tra điều kiện dừng khi gặp ô trống trong bảng băm. Các ảnh dưới đây minh họa giao diện Dashboard theo từng vị trí khóa; bảng 3 mới là số liệu C++ Core dùng để kết luận hiệu năng.

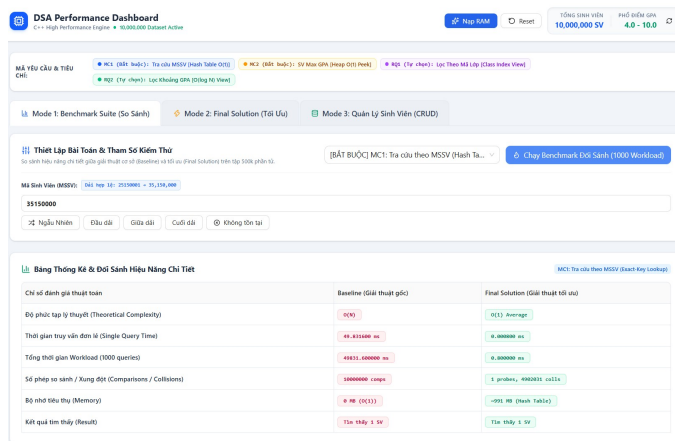
Kết quả thực nghiệm cho thấy Bảng băm đóng duy trì thời gian trả kết quả ở mức rất nhỏ và gần như không phụ thuộc vị trí khóa, trong khi thuật toán duyệt tuyến tính tăng rõ rệt khi khóa nằm sâu trong vector hoặc không tồn tại. Trên workload 1.000 truy vấn ở tập 10.000.000 sinh viên, median của Linear Search là 26424.507 ms, còn Hash Search là 0.024 ms. Do thao tác Hash lookup quá



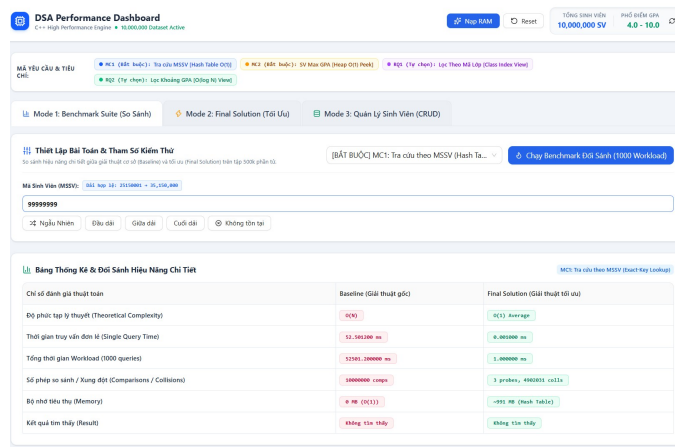
Hình 15. Benchmark MC1 tra cứu MSSV tại đầu dải dữ liệu



Hình 16. Benchmark MC1 tra cứu MSSV tại giữa dải dữ liệu
 ngắn, các con số dưới micro-giây cho một truy vấn đơn lẻ chỉ nên được hiểu như xu hướng trung bình sau lặp batch, không phải độ trễ cảm nhận của giao diện.

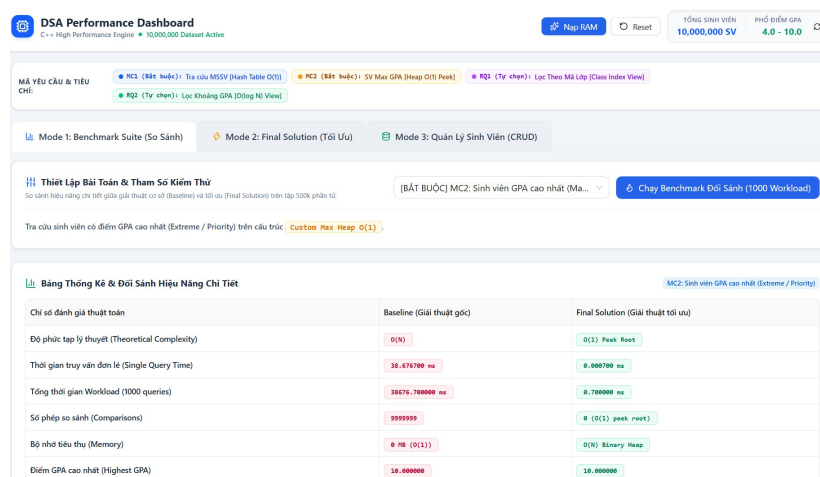


Hình 17. Benchmark MC1 tra cứu MSSV tại cuối dải dữ liệu



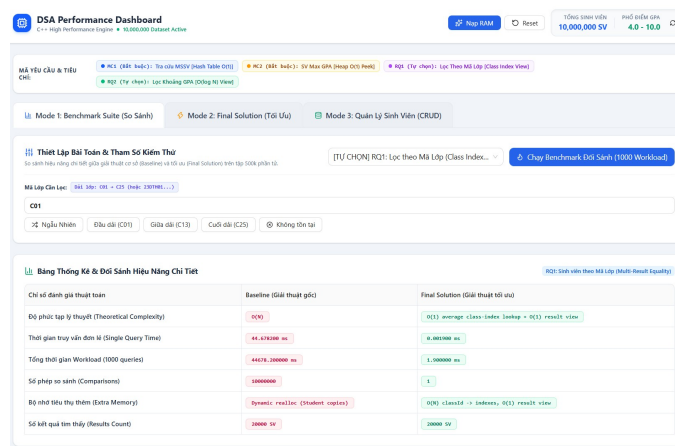
2.7.3. Kết quả đo thời gian trích xuất cực tri MC2 và xây dựng đồng

Đối với yêu cầu MC2, sau khi CustomMaxHeapGpaFinder đã xây dựng xong heap, thao tác trả sinh viên GPA cao nhất chỉ là đọc maxHeap[0], nên chi phí lý thuyết là $O(1)$. Phần đo thực nghiệm được trình bày theo hướng đối sánh: quét tuyến tính phụ thuộc N , còn heap peek gần như hằng số sau khi đã trả chi phí build $O(N)$. Trên 100 lần gọi ở tập 10.000.000 sinh viên, median của Linear Max là 3819.021 ms, trong khi Heap Max là 0.016 ms; median build heap là 1082.449 ms.

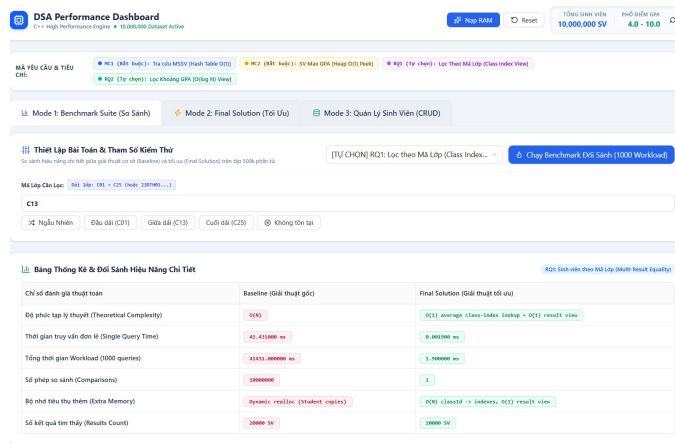


2.7.4. Kết quả đo thời gian thực thi các truy vấn tự phát hiện

Đối với yêu cầu RQ1 (Lọc danh sách sinh viên theo lớp), nhóm thực nghiệm kiểm chứng trên 4 trường hợp giao diện: lớp đầu dải (C01), lớp giữa dải (C13), lớp cuối dải (C25) và mã lớp không tồn tại (C99). Riêng bảng số C++ Core chọn lớp dày nhất trong tập 10M là 25KTP02; lớp này có 52.649 sinh viên. Baseline sao chép toàn bộ Student thỏa điều kiện, còn final dùng ClassIndexViewResult: chỉ mục classId -> indexes được build trước một lần, truy vấn chỉ còn lookup trung bình $O(1)$ và trả view không copy K kết quả.



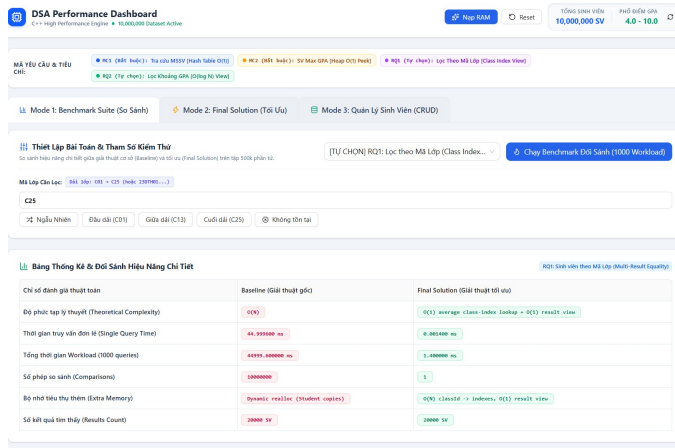
Hình 20. Benchmark RQ1 lọc danh sách sinh viên lớp C01 (Đầu dải)



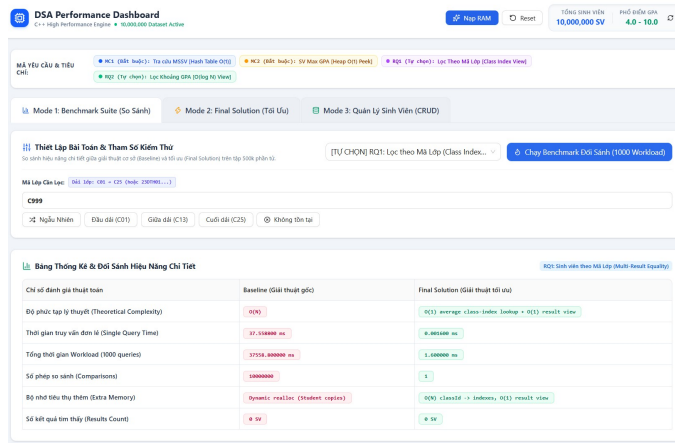
Hình 21. Benchmark RQ1 lọc danh sách sinh viên lớp C13 (Giữa dải)

Trong lần đo 10M, số phép so sánh giảm từ 10.000.000 xuống 1 lookup. Kết quả này phản ánh chi phí định vị chỉ mục và tạo view, chưa bao gồm chi phí materialize toàn bộ 52.649 kết quả nếu tầng giao diện cần xuất hết.

Đối với yêu cầu RQ2 (bảng số đo khoảng GPA [8.0, 10.0]), Binary Search hai đầu biên giúp tìm nhanh vị trí bắt đầu và kết thúc trong $O(\log N)$. Sau khi



Hình 22. Benchmark RQ1 lọc danh sách sinh viên lớp C25 (Cuối dải)



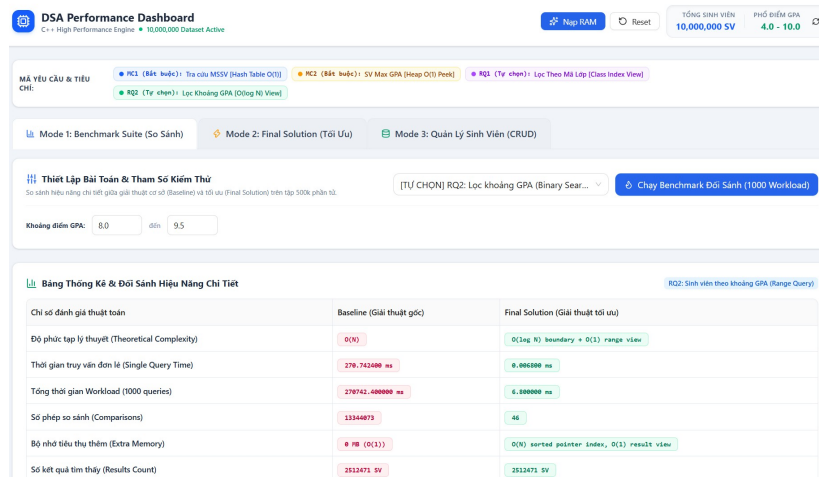
Hình 23. Benchmark RQ1 khi tra cứu mã lớp không tồn tại

tối ưu lại SortedGpaFilter theo hướng range view, code không còn sao chép K bản ghi vào `vector<Student>` kết quả. Trong lần đo C++ Core 10M, khoảng này chứa 3.343.566 sinh viên; LinearGPA mất median 331.479 ms vì vừa quét toàn bộ N vừa copy kết quả, còn SortedGPA chỉ cần 46 so sánh biên và trả view trong khoảng 0.0001 ms. Nếu tăng giao diện cần hiển thị dữ liệu, API chỉ dereference tối đa số phần tử preview cần gửi đi, nên chi phí hiển thị được tách khỏi chi phí tìm khoảng.

2.8. Đối sánh hiệu năng thực nghiệm với lý thuyết và thuật toán tuyến tính

2.8.1. So sánh tốc độ Bảng băm đóng ($O(1)$) với Quét tuyến tính ($O(N)$)

Bảng tổng hợp đối sánh thực nghiệm thời gian thực thi giữa Bảng băm đóng và Thuật toán quét tuyến tính trên workload chính được trình bày chi tiết dưới đây:



Hình 24. Dashboard minh họa truy vấn RQ2 với khoảng GPA [8.0, 9.5]; số liệu C++ Core trong Bảng 3 sử dụng workload [8.0, 10.0]

Workload	Linear	Closed Hash	Tăng tốc	Kết luận
Build cấu trúc	—	399.537 ms	—	Chi phí dựng ban đầu $O(N)$
1.000 truy vấn MSSV	26424.51 ms	0.024 ms	$\sim 1.09M\times$	Hash lookup gần như hằng số sau build
Số bước kiểm tra	$\sim 5.228B$ so sánh	1.564 probe	$\sim 3.34M\times$	Khác biệt đến từ $O(N)$ và $O(1)$ trung bình
Tính đúng	950/950 hit + 50/50 miss	950/950 hit + 50/50 miss	Tương đương	Hai phương án đều trả đúng

Số liệu thực nghiệm khẳng định rằng trên cùng workload, thời gian thực thi của Bảng băm đóng nhỏ hơn rất nhiều so với Linear Search sau khi đã trả chi phí build ban đầu. Vì thời gian hash lookup ngắn hơn độ nhiễu của đồng hồ khi đo một lần đơn lẻ, nhóm đo theo batch 1.000 truy vấn và dùng median của 9 mẫu sau 5 lượt warm-up để kết luận xu hướng $O(1)$ trung bình, không tuyên bố con số tuyệt đối như thời gian phản hồi UI.

2.8.2. So sánh tốc độ truy xuất Max-Heap ($O(1)$) với Quét tìm Max tuyến tính ($O(N)$)

Trên tập dữ liệu thực nghiệm 10.000.000 sinh viên, điểm mạnh của Max-Heap không nằm ở việc build nhanh hơn mọi phương án, mà ở chỗ sau khi build xong, thao tác đọc phần tử cực đại chỉ còn là truy cập phần tử gốc $O(1)$. Trong lần đo strict 10M, build heap mất median 1082.449 ms; sau đó 100 lần đọc cực trị chỉ mất median 0.016 ms, trong khi 100 lần quét tuyến tính mất median 3819.021 ms. Chi phí khởi tạo vẫn là $O(N)$ theo Floyd Build Heap; vì vậy báo cáo tách riêng thời gian build và thời gian query để tránh hiểu nhầm rằng heap không cần trả chi phí chuẩn bị ban đầu.

2.8.3. Đánh giá chi tiết mức độ tiêu hao bộ nhớ RAM và hiệu ứng Cache

Toàn bộ hệ thống ưu tiên các cấu trúc mảng phẳng liên tục để có lợi cho tính định vị không gian của bộ đệm CPU Cache Line 64-byte. Các con số dưới đây là ước lượng lý thuyết từ sizeof trên nền tảng 64-bit, chưa bao gồm đầy đủ allocation riêng của std::string và overhead của container.

Tập benchmark hiện tại – $N = 10.000.000$:

Vùng lưu trữ sinh viên gốc: $10.000.000 \times 104 \approx 1.040$ GB.

Bảng băm đóng: $M = 20.000.001$ ô, xấp xỉ 320 MB.

Mảng con trở sắp xếp GPA: $10.000.000 \times 8 \approx 80$ MB.

Chỉ mục ClassIndex cho RQ1: Phần chỉ số int xấp xỉ $10.000.000 \times 4 \approx 40$ MB (chưa tính overhead container).

Cây đồng cực đại: Bản sao Student xấp xỉ 1.040 GB.

Tổng footprint lý thuyết phần thân các cấu trúc chính xấp xỉ 2.52 GB, chưa bao gồm allocation riêng của std::string, bucket/metadata của container và overhead quản lý bộ nhớ của hệ điều hành.

Trong mô hình kiến trúc module hóa, các chỉ mục đọc nhiều ưu tiên mảng phẳng liên tục để tối ưu tính định vị không gian (Spatial Locality). Điểm cần lưu ý là footprint lý thuyết tính từ sizeof có thể cao hơn hoặc khác Working

Set quan sát thực tế vì còn phụ thuộc vào allocation riêng của chuỗi, overhead container và việc các module có được giữ đồng thời ở dung lượng tối đa hay không. Sau tối ưu RQ1 và RQ2, classIndex chỉ lưu các chỉ số int theo lớp, còn sortedStudents chỉ còn là mảng con trỏ; vì vậy các truy vấn nhiều kết quả không còn tạo bản sao lớn của Student trong pha query.

2.9. Nhật ký gỡ lỗi (Debug Log) và tối ưu hóa giải thuật

2.9.1. Ghi nhận lỗi phát sinh khi xử lý điều kiện dừng tìm kiếm trên Bảng băm

Trong quá trình thực nghiệm ban đầu với Bảng băm đóng, nhóm phát hiện khi tra cứu một khóa MSSV không tồn tại trong hệ thống, nếu không giới hạn số bước dò tối đa thì trong trường hợp bảng đầy hoặc vòng lặp chỉ số có sai lệch, hàm có thể lặp quá mức cần thiết. Nhóm đã hoàn thiện giải thuật: vòng dò dừng ngay lập tức khi bắt gặp ô chưa chiếm (!table[index].occupied) đầu tiên, đồng thời giới hạn số lần duyệt tối đa không vượt quá kích thước bảng M , đảm bảo thao tác tìm kiếm thất bại luôn kết thúc nhanh chóng và an toàn.

2.9.2. Tối ưu hóa cấp phát bộ nhớ và bảo toàn con trỏ trong Bulk Load

Trong quá trình nạp dữ liệu quy mô lớn, việc std::vector tự động mở rộng dung lượng nhiều lần sẽ phát sinh chi phí sao chép dữ liệu và có thể làm thay đổi địa chỉ bộ nhớ của các phần tử Student. Lệnh reserve() giúp giảm số lần tái cấp phát và giữ ổn định vùng dữ liệu gốc trước khi xây dựng các chỉ mục lưu con trỏ như HashTable và SortedGpaFilter. Riêng MaxHeap không phụ thuộc vào con trỏ về vùng gốc vì chủ động xây dựng bản sao Student nội bộ. Đánh đổi của cách cấp phát trước là có thể giữ capacity lớn hơn nhu cầu tức thời nếu tập dữ liệu thực nghiệm nhỏ hơn mức thiết kế.

2.10. Tuyên bố Trách nhiệm Giải trình về Sử dụng AI và Liêm chính Học thuật (AI-Audit Log)

Nhằm tuân thủ quy định về Liêm chính học thuật và Chuẩn mực nghiên cứu khoa học của Trường Đại học Công nghệ Kỹ thuật TP. Hồ Chí Minh, nhóm tác giả công khai nhật ký sử dụng các công cụ Trí tuệ Nhân tạo tạo sinh (GenAI / LLMs) trong quá trình thực hiện đề tài:

Hạng mục	Mức độ & Công cụ AI	Trách nhiệm và Đóng góp của Nhóm Sinh viên
Thiết kế CTDL	Gợi ý cú pháp C++20 và mẫu biểu diễn struct.	Nhóm trực tiếp thiết kế Closed Hash Table, Floyd Max-Heap, kiến trúc 3 tầng và mảng dữ liệu sắp xếp.
Cài đặt Thuật toán	Hỗ trợ rà soát cú pháp và tối ưu vòng lặp.	Tự cài đặt FNV-1a, Linear Probing, Floyd Build-Heap, iterative heapifyDown và Binary Search 2 đầu biên; triển khai Class Index View cho RQ1 bằng <code>std::unordered_map<string, vector<int>></code> ; sử dụng <code>std::sort</code> dựng mảng GPA ban đầu.
Thực nghiệm Benchmark	Hỗ trợ sinh mã kịch bản đo thời gian <code>std::chrono</code> .	Nhóm trực tiếp thực thi đo đạc trên máy tính cá nhân với tập dữ liệu hiện tại 10.000.000 sinh viên, dùng 5 lượt warm-up và 9 mẫu đo để ghi nhận số liệu thực tế.
Giao diện Web/API	Hỗ trợ khung thành phần giao diện React/Next.js.	Nhóm trực tiếp xây dựng API Bridge C++, binding dữ liệu và kiểm thử trải nghiệm người dùng.
Báo cáo LaTeX	Hỗ trợ căn chỉnh bảng biểu, phản biện tính nhất quán và định dạng.	Nhóm trực tiếp biên soạn nội dung học thuật, phân tích kết quả và kiểm duyệt toàn bộ báo cáo.

Bảng 4. Nhật ký giải trình minh bạch việc sử dụng công cụ AI (AI-Audit Log)

Cam kết liêm chính học thuật: Toàn bộ thành viên trong nhóm khẳng định

các cấu trúc dữ liệu, thuật toán và giải pháp kiến trúc trong báo cáo đều do nhóm trực tiếp nghiên cứu, lập trình và kiểm nghiệm thực tế. Công cụ AI được sử dụng để hỗ trợ rà soát cú pháp, phản biện tính nhất quán kỹ thuật và chuẩn hóa định dạng LaTeX. Nhóm sinh viên trực tiếp quyết định thiết kế, kiểm tra mã nguồn, thực thi benchmark và chịu trách nhiệm trước Hội đồng chấm thi về tính xác thực của công trình.

PHẦN KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM

1. Kết luận tổng quan về dự án

Trải qua quá trình nghiên cứu và phát triển, nhóm đã hoàn thiện hệ thống quản lý cơ sở dữ liệu sinh viên quy mô lớn và kiểm thử thực nghiệm trực tiếp tại mốc 10.000.000 bản ghi, qua đó đánh giá khả năng vận hành ở quy mô lớn cũng như bài toán cân bằng giữa tốc độ xử lý tức thì và giới hạn tài nguyên phần cứng.

Việc tự tay cài đặt hai cấu trúc dữ liệu cốt lõi cùng các thuật toán trọng tâm giúp nhóm nắm vững bản chất hoạt động ở cấp độ bộ nhớ. Các kết quả thực nghiệm với Bảng băm đóng kết hợp FNV-1a, Max-Heap, Class Index View và Mảng con trở sắp xếp cho kết quả phù hợp với xu hướng được dự đoán từ phân tích độ phức tạp $O(1)$, $O(N)$ và $O(\log N)$ cho range view. Kết quả trên tập 10.000.000 bản ghi là cơ sở để nhóm đánh giá khả năng đáp ứng ở quy mô lớn.

Đặc biệt, mô hình kiến trúc phân tầng kết hợp chiến lược tổ chức dữ liệu mảng phẳng liên tục cho thấy hệ thống có thể vận hành ổn định trong giới hạn tài nguyên bộ nhớ RAM của máy thử nghiệm. Báo cáo ưu tiên sử dụng ước lượng footprint lý thuyết ở Mục 2.8.3 để trình bày chi phí bộ nhớ, vì Working Set quan sát bởi hệ điều hành tại từng thời điểm có thể khác với tổng vùng nhớ được cấp phát/committed. Đây là cơ sở thực nghiệm giúp nhóm kết nối giữa lý thuyết thuật toán và các kỹ thuật lập trình hệ thống hiệu năng cao.

2. Bài phản tư cá nhân của 4 thành viên (Individual Reflections - D7)

2.1. Phản tư của sinh viên Bùi Tấn Phát (MSSV: 25110289)

Ở vai trò nhóm trưởng, em chịu trách nhiệm điều phối tiến độ chung, phân công nhiệm vụ và tổng hợp báo cáo kỹ thuật của nhóm. Về mặt kỹ thuật, nhiệm vụ chính của em là cài đặt Bảng băm đóng với kích thước động $M = 2N + 1$ để phục vụ yêu cầu tra cứu MC1. Trên tập benchmark 10.000.000 bản ghi, bảng có 20.000.001 ô. Trong giai đoạn đầu kiểm thử với trường hợp mã sinh viên không

tồn tại, em gặp sự cố hàm tìm kiếm bị lặp quá số bước cần thiết do chưa chặn chặt chẽ điều kiện dừng của vòng dò tuyến tính. Sau khi rà soát cơ chế của bảng băm tĩnh hai trạng thái, em đã cập nhật điều kiện dừng ngay khi gặp ô trống đầu tiên (!occupied) hoặc khi số bước dò chạm ngưỡng kích thước bảng M , nhờ đó hàm tìm kiếm luôn trả về nullptr an toàn mà không phát sinh vòng lặp vô tận. Qua phần này, em hiểu sâu sắc về công thức kỳ vọng số bước dò của Knuth và tầm quan trọng của việc kiểm soát hệ số tải $\alpha \approx 0.50$ trong thực tế. Quá trình làm đồ án giúp em nâng cao tư duy quản lý dự án, kỹ năng phân bổ công việc và phối hợp làm việc nhóm hiệu quả. Em cam kết toàn bộ phần điều phối, cài đặt và kiểm thử Bảng băm thuộc phạm vi của mình được thực hiện trung thực.

2.2. Phản tư của sinh viên Nguyễn Phan Thành Tiến (MSSV: 25110361)

Phần em đảm nhận trong đồ án là xây dựng Max-Heap tự cài đặt để trích xuất sinh viên có GPA cao nhất cho yêu cầu MC2. Quyết định kỹ thuật quan trọng nhất của em là chuyển giải thuật heapifyDown từ dạng đệ quy sang vòng lặp while nhằm loại bỏ chi phí gọi hàm và trực tiếp kiểm soát chỉ số nút con trên mảng một chiều. Khó khăn thực tế em xử lý là trường hợp nhiều sinh viên cùng đạt mức điểm cực đại 10.0; em đã thiết kế hàm so sánh ưu tiên kết hợp cả điểm GPA và so sánh chuỗi MSSV theo thứ tự từ điển để bảo đảm tính tất định ở nút gốc. Qua trải nghiệm này, em nắm vững thuật toán Floyd Build Heap $O(N)$ và cách tổ chức cây nhị phân hoàn chỉnh trên mảng liên tục. Em cam kết toàn bộ mã nguồn Max-Heap và dữ liệu thực nghiệm MC2 do em trực tiếp thực hiện.

2.3. Phản tư của sinh viên Ngô Đức Thành (MSSV: 25110335)

Nhiệm vụ trọng tâm của em trong đồ án là phụ trách yêu cầu tự phát hiện **RQ1** (Lọc danh sách sinh viên theo Lớp học phần). Để tối ưu hóa hiệu năng trên tập dữ liệu 10.000.000 bản ghi, em đã nghiên cứu và trực tiếp cài đặt cấu trúc ClassIndexFilter theo cơ chế Class Index View (`std::unordered_map<string, vector<int>`). Giải pháp này cho phép hệ thống chỉ cần duyệt mảng dữ liệu một lần duy nhất lúc khởi động để gom nhóm các chỉ số sinh viên theo mã lớp với

chi phí $O(N)$, sau đó mỗi truy vấn lọc theo lớp chỉ mất thời gian trung bình $O(1)$ để trích xuất view con trở chỉ mục mà không phải sao chép bất kỳ đối tượng Student nào. Thách thức lớn nhất em đã vượt qua là kiểm soát dung lượng bộ nhớ của chỉ mục lớp và xử lý an toàn các trường hợp mã lớp không tồn tại hoặc lớp có số lượng sinh viên lớn khi chạy tải benchmark. Qua đồ án, em nắm vững kỹ thuật lập chỉ mục phụ (Secondary Indexing) và phương pháp thiết kế cấu trúc dữ liệu không sao chép (Zero-copy View) trong C++. Em cam kết toàn bộ thuật toán và kết quả kiểm thử RQ1 thuộc phần việc của mình đều trung thực.

2.4. Phản tư của sinh viên Đặng Đoàn Minh Khang (MSSV: 25110227)

Nhiệm vụ của em trong đồ án là phụ trách yêu cầu tự phát hiện **RQ2** (Lọc sinh viên theo khoảng điểm GPA) kết hợp thiết kế kiến trúc tổng thể của hệ thống. Với RQ2, em xây dựng module SortedGpaFilter sử dụng mảng con trở `vector<const Student*>` sắp xếp theo GPA tăng dần và tự cài đặt thuật toán Tìm kiếm Nhị phân 2 đầu biên (*Lower Bound* và *Upper Bound*) để định vị chính xác khoảng giá trị $[\text{minGPA}, \text{maxGPA}]$ trong thời gian logarit $O(\log N)$. Khó khăn lớn nhất em đã xử lý là các trường hợp biên của dải điểm khi nhiều sinh viên chạm đúng giá trị cận trên maxGPA ; em đã hoàn thiện điều kiện so sánh $\leq \text{maxGPA}$ và tối ưu kết quả trả về dưới dạng range view `GpaRangeViewResult` để tránh sao chép mảng sinh viên. Bên cạnh đó, em đảm nhận vai trò thiết kế kiến trúc phân tầng 3 tầng, quy trình nạp dữ liệu hàng loạt (Bulk Load) sử dụng `reserve()` để chống phân mảnh bộ nhớ, và xây dựng tầng API Bridge kết nối C++ DSA Core với Web Dashboard trực quan. Quá trình làm đồ án giúp em nâng cao tư duy thiết kế giải thuật kết hợp kiến trúc hệ thống hiệu năng cao. Em cam kết toàn bộ phần việc thuộc phạm vi của mình được thực hiện trung thực.

TÀI LIỆU THAM KHẢO

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein (2022), *Introduction to Algorithms (CLRS)*, 4th Edition, The MIT Press & McGraw-Hill, Cambridge, Massachusetts. (Trọng tâm: Chương 6 – Heapsort và Hàng đợi ưu tiên; Chương 11 – Bảng băm và Kỹ thuật giải quyết độ Open Addressing).
- [2] Donald E. Knuth (1998), *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd Edition, Addison-Wesley Professional, Reading, Massachusetts. (Trọng tâm: Mục 6.4 – Hashing, Giải thuật Dò tuyến tính Linear Probing và Phân tích toán học về cụm sơ cấp).
- [3] Robert Sedgewick and Kevin Wayne (2011), *Algorithms*, 4th Edition, Addison-Wesley Professional, Boston. (Trọng tâm: Chương 2.4 – Priority Queues; Chương 3.4 – Hash Tables; Phân tích thực nghiệm cấu trúc dữ liệu trên bộ nhớ chính).
- [4] Mark Allen Weiss (2014), *Data Structures and Algorithm Analysis in C++*, 4th Edition, Pearson Education, Upper Saddle River, New Jersey. (Trọng tâm: Cài đặt Cấu trúc Dữ liệu tối ưu hiệu năng và Kỹ thuật quản lý con trỏ bản ghi).
- [5] Robert W. Floyd (1964), “Algorithm 245: Treesort 3”, *Communications of the ACM (CACM)*, Tập 7, Số 12, tr. 701, DOI: 10.1145/355588.365103. (Công trình gốc đề xuất giải thuật vun đống Floyd Build-Heap với độ phức tạp tuyến tính $O(N)$ từ dưới lên).
- [6] Glenn Fowler, Landon Curt Noll, and Phong Vo (2011), “Fowler–Noll–Vo Hash Function (FNV-1a)”, *Internet Engineering Task Force (IETF) Specification Draft*, Silicon Graphics & Cisco Systems. (Đặc tả giải thuật hàm băm FNV-1a 64-bit với các hằng số nguyên tố FNV_PRIME và FNV_OFFSET_BASIS).

- [7] Bjarne Stroustrup (2013), *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, Boston. (Trọng tâm: Mô hình bộ nhớ RAM, Tổ chức Layout đối tượng phẳng, Kỹ thuật tránh phân mảnh và Quản lý tài nguyên RAI).
- [8] Ulrich Drepper (2007), “What Every Programmer Should Know About Memory”, *Red Hat Technical Whitepaper*, tr. 1–114. (Phân tích chuyên sâu về kiến trúc phân cấp bộ nhớ, đường truyền Bus, bộ đệm Cache L1/L2/L3, hiện tượng Cache Line Miss và phương pháp tối ưu hóa dữ liệu liên kề Contiguous Memory Layout).
- [9] ISO/IEC 14882:2020 (2020), *Programming Languages — C++ (International Standard ISO/IEC C++20 Specification)*, International Organization for Standardization, Geneva, Switzerland. (Chuẩn ngôn ngữ lập trình C++20 và Đặc tả hệ thống thư viện thời gian chính xác cao `std::chrono`).
- [10] Đinh Bá Tiến, Trương Anh Hoàng, Trần Hạnh Nhi (2020), *Giáo trình Cấu trúc Dữ liệu và Giải thuật*, NXB Đại học Quốc gia TP. Hồ Chí Minh, TP. Hồ Chí Minh.
- [11] Nguyễn Trung Trực (2018), *Cấu trúc Dữ liệu và Giải thuật trong C/C++*, NXB Khoa học và Kỹ thuật, Hà Nội.
- [12] Bộ môn Công nghệ Phần mềm, Khoa Công nghệ Thông tin (2024), *Đề cương chi tiết và Hướng dẫn thực hành học phần Cấu trúc Dữ liệu và Giải thuật (Mã lớp học phần: DASA230179_03)*, Trường Đại học Công nghệ Kỹ thuật TP. Hồ Chí Minh.